



Casa abierta al tiempo

Universidad Autónoma Metropolitana

Unidad Iztapalapa

División de Ciencias Básicas e Ingeniería

Licenciatura en Computación

Proyecto Terminal de Investigación 1

Informe — Semana 9

Detección Inteligente de Paráfrasis mediante Modelos de Lenguaje de Gran Escala (LLM).

Aceleración del sistema en producción: arquitectura paralela, infraestructura serverless y validación del benchmark ampliado

Asesor: Dr. Benjamin Moreno Montiel

Alumno: Juan Antonio Soria Rodríguez

Matrícula: 2203041232

Grupo: CK06

Fecha: Julio 2026

Resumen

Este informe documenta la **novena semana** del proyecto. El punto de partida es el sistema v5 **desplegado en producción** en Hugging Face Spaces (<https://purplealien-deteccion-parafrasis.hf.space/>), que el asesor probó en vivo. De esa revisión salieron tres requerimientos nuevos: **R9.1**, investigar una *arquitectura paralela* que acelere el procesamiento y la comparación de datos —incluyendo si Hugging Face ofrece infraestructura *serverless* tipo AWS Lambda—; **R9.2**, investigar modelos LLM enfocados a documentación científica y la posibilidad de comparar contra un corpus científico gigante (con traducción interna español–inglés); y **R9.3**, preparar una presentación sobre los modelos del sistema (BETO, XLM-R y el ensamble, todos basados en BERT). A esto se suman cuatro tareas heredadas validadas con el alumno: ampliar el test (la palanca #1 identificada en la semana 8), el k-fold de entrenamiento, la validación multilingüe formal y el índice de oraciones al 100 %. **Las siete tareas quedaron cerradas.** R9.1: diagnóstico medido del cuello de botella (la inferencia CPU, no la búsqueda), hallazgo crítico de sobresuscripción de hilos en el contenedor de producción (corregido), mapa de las opciones *serverless* de Hugging Face, y cuantización **int8 con ONNX Runtime medida** ($2.47\times$ en el Modo 1, no-regresión aprobada, temperatura re-calibrada $T=4.35$). R9.2: investigación de encoders científicos y **prototipo federado ejecutado e integrado** (“Modo 2 federado” sobre OpenAlex con traducción interna es→en), cuya arquitectura quedó fijada por **T3**: el par cruzado es-en *colapsa* (F1 0.066) y obliga a traducir-*luego*-verificar. R9.3: presentación de 16 láminas entregada. **T2**: k-fold de entrenamiento estable ($\sigma=0.014$ — el F1 0.791 de producción no es un split con suerte). **T4**: índice de oraciones al 100 % (4.14M, HNSW-SQ8, 2.72 GB), integrado como tercera capa de recuperación. **T1**: un panel de 6 etiquetadores sin circularidad *no* alcanzó la exactitud para sustituir al humano (resultado negativo medido) y se degradó a triaje; con él se selló **test_v5** (225 pares, 33 % de positivos, el benchmark más duro del proyecto) y el encuadre pasa de doble a **triple número**. Todo se integró en el **sistema v6** (el v5 en producción queda intacto), que además fue auditado críticamente —dos correcciones de fondo: el clasificador por defecto pasa a BETO y el *Similarity Score* titular pasa a contar solo coincidencias confirmadas— y verificado de extremo a extremo (33/33 aserciones en vivo, RAM 11.7 GiB medida). Cierra la semana una re-medición de la viabilidad RAG sobre test_v5 que la descarta por cuarta vez, manteniendo la regla de integridad del proyecto: toda mejora se valida contra el benchmark robusto antes de integrarse.

Índice

1. De dónde partimos: el sistema v5 en producción	2
2. Alcance de la semana 9	2
2.1. Tareas heredadas validadas (T1–T4)	2
2.2. Requerimientos del asesor (R9.1–R9.3)	3
2.3. Plan de fases	3
3. I-1 · Arquitectura paralela y serverless en Hugging Face (R9.1)	3
3.1. Metodología de la investigación	3
3.2. Diagnóstico: dónde se va el tiempo	4
3.3. Hallazgo crítico: sobresuscripción de hilos en el Space	4
3.4. Aceleración dentro del sistema (CPU)	4
3.5. ¿Hugging Face tiene “servicios Lambda” como AWS?	5
3.6. Tres propuestas de arquitectura	6
4. Experimento de validación: perfilado y cuantización int8 (medido)	7
4.1. Diseño	7
4.2. Resultados medidos	7

4.3.	Análisis honesto: por qué los lotes no alcanzaron el 3× de la literatura	8
4.4.	Conclusiones de I-1 + experimento	8
4.5.	P-1 (quick-win) · Corrección de hilos implementada y verificada	9
4.6.	Integración del int8 en el sistema v6 y re-calibración	9
5.	I-2 · Modelos y corpus científicos + comparación cross-lingüe (R9.2)	10
5.1.	Modelos científicos: encoders, no LLMs generativos	10
5.2.	El corpus gigante: no se indexa, se consulta por API	11
5.3.	La comparación cross-lingüe: dos vías (y ya tenemos media)	11
5.4.	Arquitectura propuesta: “Modo 2 federado”	12
5.5.	Prototipo federado ejecutado (E-I2)	12
6.	T3 · Validación multilingüe: hipótesis refutada con hallazgo de diseño	12
7.	R9.3 · Presentación: BERT y los modelos del sistema	13
8.	T2 y T4 · Notebooks de GPU preparados y validados	14
8.1.	T2 · k-fold de entrenamiento (t2_kfold_entrenamiento.ipynb)	14
8.2.	T4 · Índice de oraciones al 100 % (t4_indice_oraciones.ipynb)	14
8.3.	Post-mortem del primer run de T2 y versión 2 del notebook	15
8.4.	T2 · Resultados finales (v3): el pipeline es estable	16
9.	T1 · Ampliar el test sin esfuerzo humano masivo: minado, panel y veredicto	16
9.1.	Minado de candidatos (hecho) y regla de adopción	16
9.2.	El panel sin circularidad y su validación empírica	17
9.3.	Reglas de agregación: ninguna alcanza el umbral	17
9.4.	Decisión: el panel se degrada a triaje asistente	18
9.5.	Cierre de T1: verificación humana, test_v5 sellado y evaluación	18
10.	Cierre: las cuatro mejoras pendientes implementadas en el sistema v6	19
10.1.	Mejora 3 · Umbral operativo para prevalencia realista	19
10.2.	Mejora 2 · “Modo 2 federado” integrado (R9.2)	20
10.3.	Mejora 1 · Capa de oraciones ACTIVA (T4 integrada)	20
10.4.	Mejora 4 · Votos pendientes del panel	20
10.5.	Selector de corpus en tesis completa: CBI, externo o ambos	20
10.6.	Cinco brechas de construcción corregidas + interfaz “B+”	21
11.	Auditoría crítica del sistema y dos correcciones de fondo	21
12.	KC-3 y KC-4: auditoría de las métricas del reporte de tesis con PDFs reales	22
12.1.	Cómo se calcula el Similarity Score (y por qué ~77 % es correcto)	22
13.	Resumen ejecutivo profundo: ablación de LLM y rediseño del prompt	23
13.1.	Ablación de modelos (empírica, estilo del proyecto)	23
13.2.	Rediseño del prompt y verificación en vivo	24
14.	Soporte bilingüe: analizar documentos en inglés (interno y externo)	24
14.1.	Arquitectura por superficie	25
14.2.	Costo y optimización: traducir solo los sospechosos	25
14.3.	Verificación en vivo	25

14.4. Resultados del corpus externo enriquecidos con % de similitud y métricas	26
15.Rediseño de la interfaz: dirección visual “Consola analista”	26
15.1. Sistema de diseño (ambos temas)	26
15.2. Piezas de datos como parte del sistema	27
15.3. Rediseño de la salida de resultados (claridad y comprensión)	27
16.Verificación exhaustiva del sistema v6 y viabilidad de una arquitectura RAG	28
16.1. Metodología: pruebas seguras en memoria	28
16.2. Suite exhaustiva: 33/33 aserciones pasan	28
16.3. ¿Es viable una arquitectura RAG en el sistema? Medición sobre test_v5	29
17.Pruebas con <i>ground-truth</i> y dos correcciones de atribución	30
18.Verificación pre-despliegue: pruebas de funcionamiento y auditoría de seguridad	30
19.KC-5: identificación de «posible mismo documento» en el corpus externo	31
20.Despliegue a producción del sistema v6 (autorizado por el asesor)	32
20.1. Rendimiento medido en producción	33
20.2. PWA y export a PDF verificados con navegador real	33

1. De dónde partimos: el sistema v5 en producción

La semana 9 no arranca de cero. El entregable de la semana 8 —el **sistema v5**— quedó **desplegado y funcionando en producción**:

- **URL pública:** <https://purpplealien-deteccion-parafrasis.hf.space/> (Hugging Face Spaces, plan gratuito *CPU Basic*, contenedor Docker propio).
- **Código:** GitHub privado `PurppleAlien/deteccion-parafrasis` · **Artefactos** (índices y modelos, 5.8 GB): dataset privado de Hugging Face `PurppleAlien/parafrasis-artefactos`, descargados por `start.sh` al arrancar el Space.
- **Capacidades:** doble modo (comparar pares / buscar contra el corpus), índice HNSW al 100 % (2 284 185 fragmentos, 2.4 ms por consulta), búsqueda híbrida semántica+léxica (BM25 fusionado con RRF), clasificadores XLM-R v4 calibrado ($T=5.15$), BETO v3 y ensamble, IA explicativa local, árbitro LLM opcional, reporte estilo *similarity report* con línea de tiempo, resaltado, *Flags Panel* y exportación a PDF, subida multi-formato (PDF/Word/TXT) y análisis asíncrono con barra de progreso.

Siguen vigentes los **números canónicos** y la regla del **doble número** (titular honesto sobre el test robusto; referencia sobre el test pequeño/balanceado): XLM-R v4 $F1 = 0.812$ / AUROC = 0.839 en `test_v4` (109 pares, 57 positivos); BETO v3 $F1 = 0.842$ en `test_v3_limpio`; ensamble $F1 = 0.889$ / AUROC = 0.996. Los IC95 % comunes sobre `test_v4` se solapan: los tres clasificadores son estadísticamente equivalentes.

2. Alcance de la semana 9

El alcance se definió en dos partes: la revisión de los pendientes heredados (validados con el alumno) y los requerimientos nuevos del asesor tras la demostración del sistema en producción.

2.1. Tareas heredadas validadas (T1–T4)

ID	Tarea	Motivación
T1	Ampliar el test	Palanca #1 identificada en s8: los IC95 % sobre <code>test_v4</code> son anchos (± 0.15) y solapados; la métrica está limitada por el tamaño del test (109 pares), no por el modelo. Minado por consenso + verificación humana ✓/×.
T2	k-fold de entrenamiento (E-1)	Reentrenar por folds (GPU) para reportar media $\pm \sigma$ del <i>pipeline</i> completo; complementa el k-fold de evaluación de s8.
T3	Validación multilingüe formal (E-3)	I-1 de s8 midió el idioma del corpus (es 86 % · en 6 % real); falta confirmar el rendimiento por idioma (es-es / es-en / en-en), donde XLM-R debería superar a BETO.
T4	Índice de oraciones al 100 %	Granularidad fina (detectar UNA oración copiada dentro de un párrafo original); la capa se arregló en s7 (P-5) pero su índice completo nunca se construyó. Requiere GPU; vigilar RAM.

Dos pendientes heredados quedaron **descartados** para esta semana por decisión del alumno: la infraestructura propia (TLS/auth/APK vía TWA) y la detección de texto generado por IA (se mantiene como trabajo futuro declarado).

2.2. Requerimientos del asesor (R9.1–R9.3)

Contexto: se mostró al asesor el sistema corriendo en producción y se hicieron pruebas en vivo. El asesor confirmó que *el sistema tarda en dar los resultados* y pidió:

ID	Requerimiento
R9.1	Velocidad / arquitectura paralela. Pensar e investigar una arquitectura paralela para procesar y comparar los datos dentro del sistema y hacerlo más rápido. Investigar si Hugging Face Spaces cuenta con infraestructura de servicios <i>serverless</i> tipo AWS Lambda.
R9.2	LLM / corpus científico + comparación cross-lingüe. Investigar un modelo LLM enfocado a documentación científica con corpus de documentos científicos; que el sistema pueda comparar no solo contra el corpus CBI sino contra un corpus gigante (tesis de otras universidades, artículos). Investigar si existe un LLM científico que internamente traduzca (p. ej. convertir el texto a inglés y compararlo procesado en inglés contra el corpus científico).
R9.3	Presentación sobre los modelos del sistema en producción. BERT, XLM-R y el ensamble están basados en BERT: explicar cómo usar BERT (instalación, ejecución), el caso de uso de paráfrasis, después el ensamble (que es el que mejor trabaja) y otras aplicaciones posibles (p. ej. reconocimiento de imágenes o de texto).

2.3. Plan de fases

Fase	Tareas	Recurso
Investigación (I)	I-1 arquitectura paralela + serverless (R9.1) · I-2 LLM/corpus científicos + cross-lingüe (R9.2)	CPU/web
Experimentación (E)	E-1 k-fold de entrenamiento (T2) · E-2 multilingüe (T3) · E-perfilado del cuello de botella (base de R9.1)	CPU + Colab
Datos (D)	D-1 minado + verificación humana para ampliar el test (T1)	CPU + alumno
Implementación (P)	P-1 optimización/paralelización derivada de I-1 · P-2 índice de oraciones 100 % (T4)	CPU + Colab
Presentación	PPTX de BERT / BETO / XLM-R / ensamble (R9.3)	CPU

3. I-1 · Arquitectura paralela y serverless en Hugging Face (R9.1)

3.1. Metodología de la investigación

Antes de proponer arquitecturas se fijó el **diagnóstico medido** de dónde se va el tiempo (con los datos de la semana 8), se revisó la documentación oficial de Hugging Face (Spaces, ZeroGPU,

Inference Endpoints, Inference Providers, Jobs) y la literatura publicada de aceleración de *transformers* en CPU (ONNX Runtime, cuantización dinámica int8). Todas las afirmaciones citan su fuente; las estimaciones se marcan como tales y se contrastan con mediciones propias en la sección 4.

3.2. Diagnóstico: dónde se va el tiempo

Con las mediciones de la semana 8 (análisis de una tesis de 93 000 caracteres → 464 fragmentos en ~115 s locales tras las optimizaciones del Bloque AH):

Fase	Costo	Estado
Carga del índice HNSW + BM25 (1ª vez)	~100–150 s	resuelto en s8 (pre-carga en <i>background</i>)
Búsqueda en el índice (HNSW, efSearch=64)	2.4 ms/consulta	resuelto en s8 (era 575 ms; ~240×)
Embeddings MiniLM de los fragmentos (CPU)	~30–40 % del tiempo	objetivo
Cross-encoder XLM-R/BETO en lote (CPU)	~50–60 % del tiempo	objetivo principal
Agregación y reporte (regex, spans, JSON)	despreciable	—

Diagnóstico

La lentitud que observó el asesor **no** está en la búsqueda (ya optimizada con HNSW) sino en la **inferencia de los transformers sobre CPU**: los embeddings del bi-encoder y, sobre todo, la clasificación del cross-encoder. Cualquier “arquitectura paralela” útil debe atacar esas dos fases.

3.3. Hallazgo crítico: sobresuscripción de hilos en el Space

La documentación oficial de Spaces establece que el plan gratuito (*CPU Basic*) otorga **2 vCPU** y **16 GB de RAM**, y que la cuota real se expone en la variable de entorno `CPU_CORES`. Sin embargo, nuestro `core_v4.py` fija los hilos de PyTorch con:

```
1 torch.set_num_threads(os.cpu_count() or 4)
```

Dentro del contenedor, `os.cpu_count()` reporta los núcleos del *host* físico —el log del despliegue mostró “16 vCPU”— y no la cuota del contenedor. El resultado es que el sistema en producción corre con **16 hilos compitiendo por 2 núcleos reales** (*thread thrashing*: cambio de contexto constante que *empeora* la latencia). Lo mismo aplica al `ThreadPoolExecutor(max_workers=4)` del Modo 2.

Quick win #1 (gratis)

Leer la cuota real (`CPU_CORES` o `/sys/fs/cgroup/cpu.max`) y fijar `torch.set_num_threads(cuota)`. Son ~10 líneas y solo afectan al despliegue (la máquina local ya usa sus núcleos correctos).

3.4. Aceleración dentro del sistema (CPU)

Opciones ordenadas por relación beneficio/esfuerzo, con la evidencia publicada:

A1. Hilos = cuota real (§3.3). Esfuerzo trivial, riesgo nulo.

- A2. Cuantización dinámica int8 + ONNX Runtime** (*la palanca grande*). Convertir MiniLM y el cross-encoder a ONNX y cuantizar los pesos float32→int8 con *Optimum*; **sentence-transformers** trae soporte nativo (**backend="onnx"**). Evidencia publicada: **3.08×** en CPU para sentence-transformers (documentación oficial de sbert) y **2.9×** para BERT-base con cuantización dinámica (Microsoft, ONNX Runtime), con pérdida de exactitud marginal. Aplicado a nuestras dos fases dominantes ($\sim 90\%$ del tiempo), el análisis de tesis pasaría de ~ 115 s a $\sim 40\text{--}55$ s estimados. *Cuidado honesto*: la cuantización puede mover las probabilidades del clasificador; se re-valida sobre **test_v4** (criterio de no-regresión) y se re-verifica la temperatura $T=5.15$.
- A3. Pipeline solapado** (productor–consumidor): calcular los embeddings del lote $i+1$ mientras se clasifica el lote i . Útil con ≥ 4 vCPU; marginal con 2 vCPU (ambas etapas compiten por los mismos núcleos).
- A4. Multi-proceso**: 2–8 *workers* de inferencia repartiéndose los fragmentos. Mismo caveat: escala con los núcleos disponibles.
- A5. Reducir trabajo**: ya se hizo en s8 (TOPE_EVIDENCIA 150→80, lote 16→32, banda BM25 acotada; 300→115 s). Margen restante limitado sin arriesgar *recall*.

Nota técnica: los hilos del `ThreadPoolExecutor` actual sí ayudan (PyTorch libera el GIL durante la inferencia), pero **no crean paralelismo nuevo**: reparten los mismos núcleos. La ganancia real viene de int8 (menos trabajo por token) y, con más núcleos, de A3/A4.

3.5. ¿Hugging Face tiene “servicios Lambda” como AWS?

Respuesta corta: no existe un equivalente exacto de AWS Lambda (funciones HTTP por petición, facturadas por invocación), **pero Hugging Face cubre el espectro serverless con cuatro productos**. Como referencia, AWS Lambda admite contenedores de hasta 10 GB con *timeout* máximo de 15 minutos y **no ofrece GPU** —de hecho, ni en AWS resolvería nuestra fase lenta; su análogo (SageMaker Serverless Inference) también es solo-CPU.

Producto	Qué es	¿“Tipo Lambda”?	¿Nos sirve?
ZeroGPU	GPU (NVIDIA RTX Pro 6000 Blackwell; 48 o 96 GB) asignada <i>por llamada de función</i> (<code>@spaces.GPU</code>) y liberada al terminar. PRO \$9/mes: 40 min GPU/día, extensible a \$1 por 10 min	Sí — es lo más parecido a “Lambda con GPU”	Restricción clave: solo Gradio SDK (nuestro Space es Docker). Usable como microservicio aparte (opción 2, §3.6)
Inference Providers	API por petición a modelos del catálogo; facturación por tiempo de cómputo	Sí (en facturación)	No: no ejecuta nuestro <i>pipeline</i> propio (índice + BM25 + RRF + clasificador <i>fine-tuned</i>)
Inference End-points	Despliegue dedicado de un modelo (incluye repos privados <i>fine-tuned</i>) con <i>autoscaling</i> y scale-to-zero ; desde ~\$0.03/h CPU, ~\$0.50/h GPU	Parcial (escala a cero, con <i>cold start</i>)	Viable para servir solo el cross-encoder como microservicio
Jobs	Cómputo efímero bajo demanda (<code>hf jobs run</code>), CPU→A100/TPU, pago por segundo	Sí, para <i>batch</i> (tipo AWS Batch)	Sí: ideal para trabajos puntuales, p.ej. construir el índice de oraciones (T4) sin depender de Colab
Spaces (upgrade)	El mismo Space con mejor hardware: CPU Upgrade 8 vCPU/32 GB \$0.03/h · T4 small (GPU 16 GB) \$0.40/h; el tiempo dormido no se factura	No (vertical, no serverless)	Sí: el camino sin cambiar una línea de código

3.6. Tres propuestas de arquitectura

Opción 1. Optimización in-place en CPU (recomendada como primer paso; costo \$0): hilos = cuota (A1) + modelos ONNX int8 (A2) en el mismo Space gratuito. Ganancia estimada 2.5–3× en las fases dominantes ($\sim 115\text{ s} \rightarrow \sim 40\text{--}55\text{ s}$ por tesis; Modo 1 $< 0.5\text{ s}$). Riesgo: re-validar F1/AUROC tras cuantizar (§4).

Opción 2. Arquitectura paralela distribuida con ZeroGPU (PRO \$9/mes): el Space Docker actual conserva la UI, el índice, BM25 y el reporte, y *delega la inferencia por lotes* a un segundo Space (Gradio) cuya función decorada `@spaces.GPU` recibe lotes de pares/fragmentos y devuelve probabilidades y embeddings —una GPU de 48 GB existe solo mientras corre la llamada. Es el equivalente funcional de Lambda aplicado a nuestro problema, y la materialización directa de la “arquitectura paralela” solicitada: N lotes pueden despacharse como N llamadas concurrentes. Estimación: inferencia 10–50× más rápida; el análisis de tesis quedaría dominado por la red ($\sim 10\text{--}20\text{ s}$). Esfuerzo: medio-alto (segundo Space, colas, cuota diaria de GPU).

Opción 3. Upgrade vertical del Space (dinero por velocidad, cero código): CPU Upgrade (8 vCPU, \$0.03/h) habilita A3/A4 y suma con int8 ($\sim 15\text{--}25\text{ s}$ /tesis); T4 small (\$0.40/h) usaría CUDA sin cambios ($\sim 5\text{--}10\text{ s}$ /tesis, con 15 GB de RAM justos para

el índice de 4.1 GB).

Recomendación (a discutir con el asesor): fase 1 = opción 1 (gratis, medible, sin riesgo de arquitectura), validada con el experimento de la sección 4; fase 2 = opción 2 como diseño/prototipo de la arquitectura paralela para el informe y la defensa. La opción 3 queda documentada como alternativa.

4. Experimento de validación: perfilado y cuantización int8 (medi-do)

Para no quedarnos en estimaciones de literatura, se ejecutó un experimento local que mide (i) el reparto real del tiempo por fase, (ii) el *speedup* de la cuantización int8 con ONNX Runtime sobre nuestros modelos concretos, y (iii) la **no-regresión** de calidad sobre el benchmark robusto `test_v4` (109 pares). Script: `semana_9/implementaciones/01_perfilado_onnx.py`; respaldo JSON: `semana_9/resultados/01_perfilado_onnx.json`.

4.1. Diseño

- **Carga de trabajo de referencia:** 464 fragmentos de ~ 400 caracteres (el mismo volumen del caso de referencia de s8: tesis de 93k caracteres) y el lote de 80 pares que clasifica el reporte (TOPE_EVIDENCIA), con la configuración exacta de producción (MiniLM lote 64; cross-encoder `maxlen=160`, lote 32; Modo 1 `maxlen=256`; $T=5.15$).
- **Cuantización:** XLM-R exportado a ONNX y cuantizado int8 dinámico con Optimum (`AutoQuantizationCon` MiniLM con el soporte nativo de sentence-transformers (`model Quint8_avx2.onnx`).
- **No-regresión:** evaluación completa de `test_v4` (109 pares) con fp32 y con int8, mismo protocolo de producción. Criterio pre-registrado: el F1 int8 debe caer dentro del IC95 % del fp32 ($[0.71, 0.86]$, Bloque AC de s8) y $|\Delta F1| \leq 0.05$; si no, la cuantización se descarta.
- **Entorno:** CPU local Intel i7-4770HQ (4 núcleos/8 hilos, arquitectura Haswell 2014), **con carga de fondo** (contenedores Docker activos): las cifras absolutas llevan ruido; las relaciones fp32/int8 de la misma corrida son el dato fiable, y la latencia del par suelto se reporta como mediana de 8 repeticiones.

4.2. Resultados medidos

Reproducción del baseline (control de validez): la evaluación fp32 de `test_v4` reprodujo exactamente los números de producción de la semana 8 —F1 = 0.791, AUROC = 0.819, P = 0.708, R = 0.895, FP = 21, FN = 6—, lo que confirma que el montaje del experimento es fiel al sistema desplegado.

Fase (config de producción)	fp32 (PyTorch)	int8 (ONNX)	Speedup
Modo 1 · par suelto (mediana de 8)	1586 ms	642 ms	2.47×
Cross-encoder · lote 80 pares	51.1 s	40.2 s	1.27×
MiniLM · 464 fragmentos	74.4 s	66.6 s	1.12×
Evaluación <code>test_v4</code> (ms/par)	1729 ms	729 ms	2.37×
Tamaño XLM-R	1112 MB	279 MB	4.0× menor
Tamaño MiniLM	449 MB	118 MB	3.8× menor

No-regresión de calidad (test_v4, 109 pares, $T=5.15$, $\theta=0.5$):

	F1	AUROC	P	R	FP	FN
XLM-R fp32 (producción)	0.791	0.819	0.708	0.895	21	6
XLM-R int8 (ONNX)	0.754	0.818	0.708	0.807	19	11
Δ	-0.037	-0.001	0.000	-0.088	-2	+5

Veredicto del criterio pre-registrado: APROBADA ($F1_{\text{int8}} = 0.754 \in [0.71, 0.86]$ y $|\Delta F1| = 0.037 \leq 0.05$), *con matiz*: la caída viene íntegramente del *recall* (FN 6 $\rightarrow$ 11) por un corrimiento fino de las probabilidades cerca del umbral —el AUROC es prácticamente idéntico ($\Delta = -0.001$) y el coseno medio entre los embeddings fp32 $\leftrightarrow$ int8 del MiniLM es 0.992, es decir, **el ordenamiento se conserva; lo que se mueve es la calibración**. Antes de integrar int8 a producción habría que **re-calibrar la temperatura** sobre el modelo cuantizado (mismo procedimiento P-2/s7), lo que previsiblemente recupera el recall.

4.3. Análisis honesto: por qué los lotes no alcanzaron el $3\times$ de la literatura

Hallazgo de hardware

La CPU local (i7-4770HQ, Haswell 2014) soporta **solo AVX2**: carece de **AVX512-VNNI**, el conjunto de instrucciones que acelera los productos punto int8 y en el que se apoyan los speedups de $2.9\text{--}3\times$ reportados por Microsoft y sentence-transformers. Sin VNNI, el int8 ahorra sobre todo memoria (modelos $4\times$ más chicos) y rinde bien en secuencias sueltas (Modo 1: **$2.47\times$** medido), pero gana poco en lotes grandes ($1.1\text{--}1.3\times$).

Dos consecuencias prácticas:

1. La medición local es una **cota inferior**: los servidores de Hugging Face Spaces usan CPUs de servidor modernas donde el int8 con VNNI puede acercarse al $3\times$ también en lotes. El paso siguiente natural de P-1 es medir `/proc/cpuinfo` y el benchmark en el Space real.
2. El resultado **refuerza la opción 2** (ZeroGPU) como la palanca grande de velocidad: en CPU vieja el int8 no basta para los lotes del análisis de tesis completa; la inferencia por lotes es exactamente el caso donde una GPU por llamada rinde $10\text{--}50\times$.

4.4. Conclusiones de I-1 + experimento

1. El cuello de botella del sistema es la **inferencia CPU** (no la búsqueda); dentro de ella, el análisis de tesis está dominado por el cross-encoder por lotes y los embeddings.
2. En producción hay un **bug de hilos** (16 hilos sobre 2 vCPU) cuya corrección es gratuita e inmediata (leer `CPU_CORES`).
3. La cuantización int8 **pasó la no-regresión** (AUROC intacto) y da **$2.47\times$ medido en el Modo 1** incluso en CPU antigua; para lotes, su beneficio depende de VNNI (verificar en el Space). Integración condicionada a re-calibrar T sobre el modelo cuantizado.
4. Hugging Face no tiene un “Lambda” exacto; el equivalente funcional para nuestro caso es **ZeroGPU** (GPU por llamada, \$9/mes PRO), viable como microservicio de inferencia junto al Space Docker actual, y **Jobs** (pago por segundo) para trabajos batch como el índice de oraciones (T4).

- Recomendación al asesor: fase 1 = quick-win de hilos + int8 *con re-calibración* y medición en el Space; fase 2 = prototipo de la arquitectura paralela distribuida con ZeroGPU.

Respaldo: `semana_9/resultados/01_perfilado_onnx.json` · script: `semana_9/implementaciones/01_perfi`
 · modelos exportados: `semana_9/resultados/onnx/`.

4.5. P-1 (quick-win) · Corrección de hilos implementada y verificada

Regla de trabajo de la semana 9: el entregable de la semana 8 (`semana_8/sistema_v5`, desplegado en producción) queda **congelado** mientras el asesor lo revisa junto con su documentación; toda mejora de esta semana se hace en el **sistema v6** (`semana_9/sistema_v6/`, copia de v5 con los modelos e índices compartidos por enlaces simbólicos), siguiendo el mismo patrón v4→v5 de la semana 8. El despliegue del v6 se hará al final de la semana, cuando el asesor dé sus indicaciones.

El *quick-win* de la sección 3.3 quedó **implementado** en `semana_9/sistema_v6/core_v4.py`:

- Nueva función `_nucleos_reales()`: lee la cuota real del contenedor en este orden — cgroup v2 (`/sys/fs/cgroup/cpu.max`), cgroup v1 (`cpu.cfs_quota_us/cpu.cfs_period_us`), la variable `CPU_CORES` documentada por Hugging Face, y como último recurso `os.cpu_count()`.
- `torch.set_num_threads(NUCLEOS)` usa esa cuota, y los *workers* del cross-encoder del Modo 2 pasan de un valor fijo (4) a `WORKERS_CE = max(2, min(4, NUCLEOS))`.
- Nueva función `_tiene_vnni()`: el arranque ahora imprime una línea de diagnóstico — `CPU threads = N` (cuota real; host reporta M) · AVX512-VNNI: *sí/no* — de modo que el **propio log del Space responderá** cuántos núcleos reales hay y si su CPU soporta VNNI (la incógnita de la sección anterior).

Verificación local (en sistema v6): (i) compila; (ii) en la máquina local detecta 8 núcleos (sin cuota cgroup → `os.cpu_count()`); (iii) simulando el entorno del Space (`CPU_CORES=2`) detecta 2 núcleos y 2 *workers*; (iv) prueba funcional del Modo 1 correcta (clasificación y probabilidades intactas — el cambio no toca el modelo, solo el número de hilos). **Pendiente:** desplegar el sistema v6 al final de la semana (tras las indicaciones del asesor) para que el fix actúe en producción y su log confirme cuota y VNNI reales.

4.6. Integración del int8 en el sistema v6 y re-calibración

El experimento de la §4 dejó el int8 *aprobado condicionado a re-calibrar la temperatura* sobre el modelo cuantizado. Cumplido (`09_recalibrar_int8.py`, mismo protocolo que s7 — NLL mínimo sobre `test_v4`):

	int8 sin calibrar	int8 ($T=4.35$, NLL)	int8 con $T=5.15$ (la T del fp32)
ECE	0.259	0.083	0.065
F1 / AUROC (<code>test_v4</code>)	0.754 / 0.818	0.754 / 0.818	0.754 / 0.818

Las tres columnas son el modelo *int8* (respaldo: `09_recalibracion_int8.json`); F1 y AUROC son invariantes a T (reescala monótona). La referencia fp32 se midió aparte: F1 = 0.791 / AUROC = 0.819 en el mismo experimento de la §4, y ECE = 0.076 con $T=5.15$ (`p2_calibracion_xlmr.json`, s7).

La T óptima del int8 **por NLL** (el protocolo declarado, idéntico al de s7) es **4.35**, distinta de la 5.15 del fp32: la cuantización sí desplaza los logits, de modo que la condición de re-calibrar era

correcta y no un formalismo. *Matiz honesto*: por la métrica puntual ECE, el int8 con la T heredada mediría algo menos (0.065 vs 0.083) — NLL y ECE no tienen por qué coincidir; se conserva el criterio NLL por consistencia con el protocolo de s7 y se declara la diferencia. La degradación agregada del int8 (−0.037 F1, −0.001 AUROC frente al fp32 del mismo experimento) es la ya aprobada. *Nota honesta*: aunque la paridad agregada se sostiene, pares individuales pueden cambiar de clase (t004 de test_v4: $\Delta\text{logit fp32} +3.17 \rightarrow \text{int8} -1.61$) — mitigado por la red de seguridad “alta similitud — verificar” y por el ensamble, que incluye BETO fp32 con recall 1.0.

Integración (solo semana_9/sistema_v6): el clasificador (_Modelo en core_v4.py y el CLI clasificador_modol.py) detecta <ruta>-onnx-int8/ y, si existe onnxruntime, usa ese backend con su propia calibracion.json; ONNX_INT8=0 lo desactiva y **cualquier fallo degrada en silencio a torch fp32**. Verificado de punta a punta (XLM-R int8, BETO torch, ensamble mixto). Latencia local (i7, 8 hilos, par corto): 139 ms int8 vs 262 ms fp32; en el Space (2 vCPU, y a la espera de saber si tiene VNNI) se medirá al desplegar.

Tarjeta de métricas “triple número”. Las tres capas del sistema (core_v4.METRICAS_VALIDACION, app_unificada._MET y el frontend metricsCard) se sincronizan para mostrar ahora *tres* contextos por modelo —fácil-histórico, robusto-balanceado y el nuevo **test_v5 realista-difícil**— con la explicación en pantalla de por qué el F1 baja mientras el AUROC se mantiene. Es la aplicación directa, en producción, del encuadre honesto de la §9.5.

5. I-2 · Modelos y corpus científicos + comparación cross-lingüe (R9.2)

Documento fuente: semana_9/investigacion/i2_llm_corpus_cientifico.md. La pregunta del asesor tiene tres partes: un modelo enfocado a documentación científica, la comparación contra un *corpus gigante* (tesis de otras universidades, artículos) y la posibilidad de que el sistema “traduzca internamente” a inglés para comparar contra ese corpus.

5.1. Modelos científicos: encoders, no LLMs generativos

Para *comparación y similitud*, el estado del arte en NLP científico no son LLMs generativos sino **encoders de documentos científicos** —la misma familia BERT de nuestro sistema—:

Modelo	Qué es	Limitación
SciBERT (2019)	BERT pre-entrenado en 1.14M papers	Encoder general; solo inglés
SPECTER2 (2023)	Embeddings de documentos científicos; 6M tripletas de citas, 23 campos; el mejor del benchmark SciRepEval (65.46 vs SciNCL 65.08 vs SciBERT 57.42)	Solo inglés
SciNCL (2022)	Contrastivo sobre el grafo de citas	$\approx$ SPECTER2; solo inglés
Galactica (2022)	LLM generativo científico (Meta)	Retirado en 3 días (alucinaciones); no aplica

Lectura para el proyecto

El “LLM científico” aplicado a similitud se materializa como **SPECTER2** (embeddings científicos), cuya restricción central es el **inglés** — exactamente el motivo de la tercera parte de la pregunta del asesor.

5.2. El corpus gigante: no se indexa, se consulta por API

Indexar 200–260M de obras localmente es inviable (solo los embeddings 384d en float32 de 250M documentos $\approx$ **384 GB**; nuestro índice CBI de 2.28M fragmentos ya ocupa 3.5–4.1 GB). La vía práctica es consumir las **APIs públicas**:

Fuente	Tamaño	Acceso	Qué aporta
OpenAlex	250M+ obras	API gratuita sin key (100k req/día)	metadata + abstract; verificado hoy : consulta de prueba $\rightarrow$ 34 729 resultados con abstract
Semantic Scholar	200M+ papers	API gratuita con key (el pool sin key devolvió 429/500 hoy — verificado)	metadata, abstract y el embedding SPECTER2 precalculado de cada paper vía API
CORE	260M+ registros	API con key gratuita	$\sim$ 32M full-text open access
La Referencia	red de 10 países	OAI-PMH / portal	tesis de 100+ universidades latinoamericanas — lo más cercano a “tesis de otras universidades”
Dialnet / SciELO / Redalyc	mayor base en español	portal / OAI-PMH	tesis doctorales españolas; artículos OA iberoamericanos

Hallazgo que cambia la arquitectura

Semantic Scholar ya sirve el **embedding SPECTER2 de cada paper vía API**: no hay que indexar nada localmente. El “corpus gigante” se convierte en una **capa de recuperación remota** montada sobre el patrón recuperar $\rightarrow$ clasificar que el sistema ya tiene.

5.3. La comparación cross-lingüe: dos vías (y ya tenemos media)

La idea de “traducir internamente y comparar en inglés” tiene dos materializaciones:

- **Vía A — traducir y comparar** (*translate-then-embed*): es $\rightarrow$ MarianMT/NLLB $\rightarrow$ en $\rightarrow$ SPECTER2. Aprovecha los modelos científicos en inglés tal cual (MarianMT ya lo usamos en s5–s7), pero duplica inferencia y añade el ruido de la traducción; la evidencia publicada reporta que el enfoque traducir-luego-procesar es **20–30 $\times$ más lento** que el multilingüe directo (Meta Engineering).
- **Vía B — embeddings cross-lingües** (espacio compartido, *sin* traducir): LaBSE (109 idiomas) y multilingual-E5 (100 idiomas; supera a LaBSE en *retrieval*) mapean español e inglés al mismo espacio vectorial — la comparación es $\leftrightarrow$ en es un coseno directo. **Nuestro stack ya es de esta familia**: el MiniLM de recuperación es *paraphrase-multilingual* y el cross-encoder **XLM-R v4 es multilingüe** (puede verificar pares es-en directamente) — la elección de backbone de la semana 7 resultó presciente para R9.2.

Síntesis honesta: no existe un “LLM científico que traduce internamente” como producto único; existen la tubería (A) y los encoders multilingües (B). Propuesta: prototipo con **B** (ya tenemos los

modelos) y **A** como brazo de comparación — una ablación medida, el formato que el proyecto domina. La tarea heredada **T3** (validación multilingüe es-es/es-en/en-en) pasa de tarea suelta a **prerequisito medido** de esta capa.

5.4. Arquitectura propuesta: “Modo 2 federado”

Capa	Función
Local (existente)	HNSW+BM25 sobre CBI (2.28M frags) → cross-encoder → reporte
Remota (nueva)	(1) traducción es→en (MarianMT); (2) consulta a OpenAlex (sin key, ~250M obras); (3) candidatos título+abstract verificados en-en por beto-pawsx (T3: el par cross-lingüe colapsa → se traduce antes); (4) sección nueva del reporte: “fuentes externas (literatura científica)”

Limitación declarada: la capa remota compara contra título+abstract (full-text solo en el subconjunto OA de CORE) — detecta cercanía temática/parafraseo de ideas frente a la literatura, no plagio literal de cuerpos completos. **Costo:** \$0 (keys gratuitas).

5.5. Prototipo federado ejecutado (E-I2)

El prototipo se construyó y corrió de extremo a extremo (`10_i2_prototipo_federado.py`), con la arquitectura que **T3 validó** (§6, traducir-luego-verificar):

frag es $\xrightarrow{\text{MarianMT es} \rightarrow \text{en}}$ frag en $\xrightarrow{\text{OpenAlex (API)}}$ abstracts $\xrightarrow{\text{beto-pawsx en-en}}$ prob de solapamiento

Tres decisiones, todas justificadas: **(1)** el corpus externo *no se indexa* (250M *abstracts* no caben en disco/RAM) sino que se consulta **por API bajo demanda** — encaja con el modelo serverless de R9.1; **(2)** se verifica **en-en**, no es-en, porque T3 demostró que el par cross-lingüe *colapsa* (F1 0.066) mientras en-en funciona — de ahí traducir *antes* de verificar; **(3)** el verificador es **beto-pawsx** (ajeno al gold del proyecto), sin circularidad. Se sustituyó Semantic Scholar (429 sin key) por **OpenAlex**, que funciona *sin API key* (basta `mailto` en la consulta) sobre ~250M obras.

Resultado (3 fragmentos, red viva): OpenAlex recuperó literatura temáticamente correcta (lecho fluidizado → *Chemical Reviews*; plegamiento de proteínas → *Protein Science* 1995; POO → *ACM Computing Surveys* 1988), y las probabilidades de solapamiento salieron **bajas** (≤ 0.028): correcto y honesto — los fragmentos son *afines de tema*, no paráfrasis de esos *abstracts*, y el verificador **no da falsos positivos**. Control de sanidad: ante una paráfrasis en-en real el mismo verificador da **0.994**, y ante no-relación **0.007** — la cadena discrimina, las probabilidades bajas son la respuesta correcta. Queda demostrada la viabilidad de R9.2 de extremo a extremo; la integración como “Modo 2 federado” seleccionable en el v6 es el siguiente paso natural, sujeto a las indicaciones del asesor.

6. T3 · Validación multilingüe: hipótesis refutada con hallazgo de diseño

T3 es el prerequisite medido de la capa federada (R9.2): ¿los clasificadores sostienen su rendimiento cuando uno o ambos textos están en inglés? Se tradujo `test_v4` completo con MarianMT es→en (el traductor que el proyecto usa desde s5; **la traducción se declara** — mide capacidad cross-lingüe sobre nuestro dominio con estilo MT) y se evaluaron XLM-R v4 y BETO v3 en tres variantes con el protocolo de producción. Script: `03_t3_multilingue.py`; respaldo: `t3_multilingue.json` (+ cache reproducible de traducciones).

Variante	XLM-R v4 (F1 / AUROC)	BETO v3 (F1 / AUROC)
es-es (control)	0.791 / 0.819 (reproduce producción)	0.820 / 0.828
es-en (cruzado)	0.066 / 0.420	0.067 / 0.677
en-en	0.726 / 0.734	0.771 / 0.817

Hallazgo de T3: el problema no es el idioma, es el CRUCE

La hipótesis simple (“XLM-R multilingüe sostiene; BETO solo-español se cae”) quedó **refutada**: ambos modelos funcionan razonablemente en inglés puro (en-en) — incluso BETO, por los cognados técnicos del vocabulario científico y la estructura paralela del MT — pero **ambos colapsan en el par cruzado es-en** (XLM-R predice SÍ en solo 4 de 109 pares; AUROC 0.42, peor que el azar). El *fine-tuning* con pares es-es no enseña a emparejar *a través* de idiomas: la representación subyacente es multilingüe, pero la cabeza clasificadora no.

Implicación directa para la capa federada (R9.2): la verificación es↔en directa con el cross-encoder actual queda **descartada con datos**. La ruta correcta es: recuperación cross-lingüe (MiniLM/mE5 — pendiente de medir en el prototipo E-I2) y **traducir la consulta es→en para verificar en-en** (F1 0.73–0.77 utilizable) — es decir, la “*traducción interna*” que sugirió el asesor queda **revalidada empíricamente** exactamente en el paso donde hace falta. Alternativa de mayor alcance (trabajo futuro): *fine-tuning* del verificador con pares mixtos es-en. *Caveats declarados*: el inglés es MT (el mismo traductor en ambos lados homogeneiza el estilo; un en-en nativo sería algo más difícil) y este experimento mide el verificador, no el recuperador.

7. R9.3 · Presentación: BERT y los modelos del sistema

Entregada la presentación solicitada por el asesor sobre los modelos del sistema en producción: `semana_9/presentacion/presentacion_bert_semana9.pptx` (**16 láminas**, 16:9, estilo UAM de semanas previas, con notas del orador en primera persona embebidas) + `GUION_PRESENTACION.md` (guion, regla de oro de números y 5 preguntas probables con respuesta) + 5 diagramas propios generados con matplotlib (`_gen_graficas.py`): arquitectura de BERT, cross-encoder de paráfrasis, ensamble soft-voting, comparativa con IC95 % y línea de tiempo BERT→sistema.

Estructura (sigue el orden pedido por el asesor): (1–4) qué es BERT y cómo funciona (bidi-reccional, MLM, pre-entrenamiento vs fine-tuning); (5–7) **cómo usarlo**: instalación (`pip install transformers torch`), ejecución mínima (`pipeline fill-mask` con BETO) y el fine-tuning real de pares del proyecto (Trainer + early stopping, 10–20 min en Colab T4); (8–9) **BETO** (español, gold humano, F1 0.842 canónico; fortaleza: recall) y **XLM-R** (multilingüe justificado por la medición de idioma 86/6 %, calibrado $T=5.15$, honesto 0.812/0.839 en `test_v4`; fortaleza: precisión); (10) el caso de uso de paráfrasis (cross-encoder + 3 señales → 6 tipos); (11–12) **el ensamble** (soft-voting; funciona porque los modelos fallan en pares distintos; AUROC 0.996–0.999) y la comparativa honesta (mismo test + IC95 % solapados → estadísticamente equivalentes; el valor está en el *perfil*); (13–14) **otras aplicaciones**: texto (clasificación, NER, QA, búsqueda semántica — nuestro MiniLM) e **imágenes/reconocimiento de texto** (ViT, BEiT, **TrOCR** — con conexión directa al proyecto: las 107 tesis escaneadas hoy ilegibles serían recuperables con OCR transformer, declarado como trabajo futuro); (15–16) conclusiones y cierre. Todos los números siguen la regla del doble número de la auditoría.

8. T2 y T4 · Notebooks de GPU preparados y validados

Las dos tareas heredadas que requieren GPU quedaron listas como notebooks de Colab (`semana_9/notebooks_co` generados reproduciblemente por `_gen_*.py`, documentados con el estándar I-1 y con las celdas verificadas sintácticamente).

8.1. T2 · k-fold de entrenamiento (`t2_kfold_entrenamiento.ipynb`)

Responde el R2 del asesor (s7) que quedó pendiente: la estabilidad del *pipeline completo* de entrenamiento, no solo de la evaluación (que ya se midió en s8).

- **k=5 con group-split por tesis vía union-find**: los pares que comparten cualquier documento (`doc_a↔doc_b`) quedan en el mismo grupo — cero tesis compartidas entre train y validación de cada fold.
- **Misma receta que producción** (xlm-roberta-base, 6 épocas, lote 8, `maxlen` 256, *early stopping*) sobre `gold_ampliado_train` (462) → comparable con el modelo desplegado ($F1 = 0.791$ en `test_v4`).
- **Brazo 70-30** (el esquema sugerido por el asesor) con el mismo group-split.
- Cada fold se evalúa en su validación *y* en `test_v4` sellado; salida: media $\pm\sigma$, tabla comparativa, gráficas y JSON de respaldo. Anti-fuga verificado dentro del propio notebook.

8.2. T4 · Índice de oraciones al 100 % (`t4_indice_oraciones.ipynb`)

Construye la capa de granularidad fina pendiente desde s6/s7 (el fix P-5 dejó el formato especificado pero el índice nunca se construyó):

- **Fuente**: `corpus_v4_id_map.jsonl.gz` — su orden ES el `faiss_id` del índice principal, así que el mapeo oración→párrafo padre sale directo (formato P-5).
- **Anti-duplicados por zona única**: los fragmentos se solapan (400 chars, *stride* 200) → solo se toman oraciones que empiezan en los primeros 200 caracteres; el último fragmento de cada tesis usa su longitud completa.
- **HNSW+SQ8** (vectores cuantizados a 8 bits): ~3 GB en vez de ~8 GB en fp32 — cabe junto al índice principal en los 15–16 GB de RAM disponibles. Mismo MiniLM → espacio vectorial compatible.

Validación local antes de gastar GPU (sobre datos reales): la lógica de extracción corrida sobre 3000 fragmentos reales produjo 5671 oraciones con solo **0.2 % de duplicados** (la zona única funciona) y ratio 1.89 oraciones/fragmento → extrapolación ~**4.3M oraciones** para el corpus completo; y la API `faiss.IndexHNSWSQ` se probó de extremo a extremo (auto-recuperación 10/10, similitud 1.000). **Nota de integración**: cargar esta capa en el sistema v6 será *opt-in* (variable `ORACIONES=1`) — la RAM queda justa y debe medirse antes de activarla en el Space.

T4 · Resultados finales (ejecutado en Colab, notebook v2 reanudable)

El primer intento murió cuando Colab desconectó el entorno tras ~1 h y el diseño en-memoria perdió todo el trabajo; el notebook se rediseñó como **v2 reanudable** (checkpoint por fase en Drive: cache de extracción, un `.npy` fp16 por cada chunk de 200k embeddings, e índice construible sin GPU), de modo que una desconexión nunca cuesta más que el chunk en curso. La segunda corrida completó:

Oraciones indexadas	4 144 019	(extrapolación local: $\sim 4.3\text{M}$ ✓)
Índice HNSW32-SQ8 (producto interno)	2.72 GB	(vs ~ 8 GB fp32)
id_map oración→párrafo padre (formato P-5)		177 MB
Auto-recuperación top-1 (muestra de 20)		20/20

La cuantización SQ8 quedó **validada**: un tercio del tamaño con auto-recuperación perfecta. La integración en el sistema v6 queda como *opt-in* declarado (`ORACIONES=1`) pendiente de medir la RAM total junto al índice principal.

8.3. Post-mortem del primer run de T2 y versión 2 del notebook

El primer run de T2 en Colab **colapsó** ($F1 = 0.0$ en los 5 folds y en el brazo 70-30; AUROC de validación indefinido). El diagnóstico local con los datos reales encontró **dos defectos del notebook, no del modelo**: (1) la etiqueta de `gold_ampliado_train.json` se llama y (0/1) y el notebook leía `etiqueta` → se leyeron **0 positivos de 100** y el modelo entrenó con todo en NO (colapso a predecir NO, incluso en `test_v4`); (2) ese archivo **no conserva** `doc_a/doc_b` → el agrupamiento por tesis degeneró en 462 singletons y las validaciones quedaron con una sola clase.

La lección es de la misma familia que los hallazgos C.2.x de la auditoría: *el notebook asumió el esquema sin verificarlo y carecía de asserts de sanidad* — un `assert positivos==100` habría abortado en segundos.

Reparación (v2, verificada localmente con los datos reales): se reconstruyeron los grupos de tesis por *join* de textos contra las fuentes que sí conservan el documento (`silver_consenso.json`, `positivos_humanos_limpios.json`) — **66 % de pares con doc** (303/462), 388 grupos, mayor componente de 9 pares; el 34 % restante queda como singleton (**limitación declarada**). El notebook v2 usa un lector de etiqueta robusto, **asserts de sanidad** (positivos, fuga, clases por fold, cero grupos compartidos), `StratifiedGroupKFold`, y desactiva fp16 (inestabilidad numérica conocida de XLM-R). La simulación local de los folds con los datos reales da: 5 folds $\times$ (train 80 SÍ · val 20 SÍ · 0 grupos compartidos).

Run 2 (v2): colapso parcial → **segunda causa raíz**. La re-ejecución dio un resultado mixto y revelador: los folds 1–2 sanos ($F1$ val 0.826/0.875; `test_v4` **0.800/0.797** — reproduce el 0.791 de producción, validando datos, grupos y protocolo), pero los folds 3–5 y el brazo 70-30 colapsaron a $F1 = 0$ *con etiquetas correctas* (76–85 positivos por train, asserts en verde) y con duraciones menores (el *early stopping* cortó un modelo que nunca despegó). Es el **colapso estocástico** conocido del *fine-tuning* de XLM-R con datos pequeños. Comparando contra el notebook que sí funcionó (E-4, s8), la diferencia son los **hiperparámetros**: la receta real usa `lr=2e-5`, `warmup_ratio=0.1`, `weight_decay=0.01` y paciencia 3, mientras el notebook v2 corría con los valores por omisión del `Trainer` (`lr=5e-5`, sin *warmup*) — con `lr` alto y sin calentamiento, XLM-R cae al mínimo local “todo-NO” en ~ 4 de 6 corridas.

Hallazgo metodológico (run 2)

“Reproducir la receta” debe significar **hiperparámetros exactos**, no solo arquitectura y datos: el paso de `lr 2e-5`→`5e-5` (default) y la ausencia de *warmup* convirtieron un pipeline estable en uno que colapsa estocásticamente en 2/3 de las corridas. Es la misma familia del *confounding* de E-4 v1 en s7 (lote 16 vs 8).

v3 (final): receta exacta de producción (`lr 2e-5`, *warmup* 10 %, `wd 0.01`, paciencia 3) más un **guard anti-colapso declarado**: si el $F1$ de validación sale en 0 con positivos presentes, se reintenta con otra semilla (máx. 2) y el número de reintentos **se reporta** en el JSON de resultados.

8.4. T2 · Resultados finales (v3): el pipeline es estable

La versión 3 completó las 5 particiones **sanas** (respaldo: `semana_9/resultados/resultados_kfold_t2_v3/`):

Fold	val F1	test_v4 F1	test_v4 AUROC	reintentos
1	0.818	0.787	0.847	0
2	0.857	0.793	0.842	0
3	0.849	0.803	0.851	1 (guard)
4	0.909	0.771	0.790	0
5	0.844	0.764	0.849	0
k-fold $\mu \pm \sigma$	0.856 $\pm$ 0.030	0.784 $\pm$ 0.014	0.836 $\pm$ 0.023	2 en total
70-30 (asesor)	0.852	0.772	0.848	1 (guard)
Desplegado (referencia, 1 split)	—	0.791	0.819	—

Conclusiones de T2

(1) **El pipeline es estable:** $\sigma=0.014$ en `test_v4`; el modelo desplegado ($F1 = 0.791$) cae *dentro* de la banda k-fold $[0.769, 0.798]$ — el número de producción **no es un split con suerte**, que era exactamente la duda del R2. (2) **El optimismo del CV interno queda cuantificado:** val 0.856 vs test 0.784 (~ 0.07) — el gold es más fácil que los positivos difíciles del test robusto. (3) **El 70-30 sugerido por el asesor es una aproximación válida** (0.772, dentro del rango k-fold), solo que sin la incertidumbre que aporta el k-fold. (4) **El colapso estocástico de XLM-R es real aun con la receta correcta** (~ 1 de 4 corridas: 2 reintentos del guard en 7 entrenamientos) y queda manejado y *reportado*, no oculto.

9. T1 · Ampliar el test sin esfuerzo humano masivo: minado, panel y veredicto

La tarea heredada T1 (ampliar el test robusto) arrastra el cuello de botella señalado desde s8: cada par nuevo requiere verificación humana, y escribir/verificar a mano toma horas. La pregunta de esta semana fue directa: *¿existe una metodología que amplíe el test con buenos resultados sin tanto trabajo manual?* La investigación (`investigacion/i3_ampliar_test_sin_esfuerzo.md`) identificó tres piezas con respaldo en la literatura: (a) un panel de etiquetadores automáticos *sin circularidad*, (b) *Prediction-Powered Inference* (PPI/PPI++, Angelopoulos et al. 2023) como paraguas estadístico para mezclar etiquetas automáticas con pocas humanas manteniendo intervalos válidos, y (c) *Active Testing* (Kossen et al., ICML 2021): concentrar la etiqueta humana en los puntos más informativos (los desacuerdos).

9.1. Minado de candidatos (hecho) y regla de adopción

El script `04_t1_minar_test_v5.py` minó **120 candidatos** del corpus real (HNSW + BM25, bandas de similitud altas) con **anti-fuga total**: 1927 textos y 968 pares de todos los conjuntos históricos (train, tests, silver) quedaron vetados. Antes de usar cualquier panel se fijó una regla de adopción *a priori*: **el panel solo sustituye al humano si su exactitud, medida contra etiquetas humanas reales, es ≥ 97 %**.

9.2. El panel sin circularidad y su validación empírica

La restricción central: **ningún modelo evaluado del sistema puede votar** (si BETO v3 o XLM-R etiquetan el test, después los evaluaríamos contra sus propias opiniones). El panel (script 05_t1_validar_panel.py): **4 LLMs** vía Groq (Llama-3.3-70B, Qwen3-32B, GPT-OSS-120B, Llama-4-Scout-17B) + **2 especialistas ajenos al gold: beto-pawsx** (s6, entrenado solo en PAWS-X) y mDeBERTa-v3-XNLI usando el criterio clásico de *entailment bidireccional* ($a \models b \wedge b \models a$). Se validó contra **294 etiquetas humanas históricas deduplicadas** del propio proyecto (test_v4: 109 · e5b s7: 38 · desacuerdos s7: 27 · E-4 s8: 120; 107 SÍ / 187 NO) — 1 764 votos.

Votante	κ	acc	P(SÍ)	R(SÍ)
GPT-OSS-120B	0.560	0.784	0.672	0.832
Llama-3.3-70B	0.523	0.764	0.650	0.816
Llama-4-Scout-17B	0.496	0.769	0.689	0.664
Qwen3-32B	0.474	0.785	0.821	0.484
beto-pawsx	0.300	0.721	0.838	0.290
mDeBERTa-XNLI	0.122	0.667	0.765	0.121

El perfil es nítido y complementario: los LLMs son *liberales* (recall alto, precisión media); los especialistas son *conservadores* (precisión alta, recall bajísimo — el entailment bidireccional resulta demasiado estricto para fragmentos de ~ 400 caracteres).

9.3. Reglas de agregación: ninguna alcanza el umbral

Regla	Auto	Exactitud	Exact. SÍ	Exact. NO
Unánime LLM (≥ 3 votos)	65.0 %	87.96 %	78.7 % ($n=61$)	92.3 %
Unánime LLM 4/4	53.1 %	90.38 %	85.1 %	92.7 %
Mayoría LLM 3/4	80.3 %	83.90 %	76.7 %	88.0 %
Unánime LLM + pawsx	46.9 %	94.20 %	100.0 % ($n=13$)	93.6 %
Unánime LLM + XNLI	47.3 %	91.37 %	80.0 %	92.2 %
Unánime todos (6)	43.2 %	93.70 %	100.0 % ($n=3$)	93.5 %
Asimétrica: solo NO unánime	44.2 %	92.31 %	—	92.3 %
Asimétrica: NO unánime + pawsx	42.5 %	93.60 %	—	93.6 %

Veredicto de T1: el panel NO se adopta como etiquetador autónomo

(1) **El precedente interno no generaliza.** En s7 la regla “unánime” logró 100 % de exactitud sobre 40 pares; sobre estas 294 etiquetas —mucho más adversarias, con los negativos difíciles de E-4 y los candidatos duros de e5b— la unanimidad LLM cae a 88 %. Es la misma lección del “doble número” del Modo 1: los tests fáciles inflan. (2) **La regla asimétrica queda refutada.** Automatizar solo los NO parecía seguro (los LLMs sobre-predicen SÍ), pero el NO-unánime acierta solo 92.3 %: existen ~ 10 positivos genuinos —paráfrasis con bajo solape léxico, exactamente las más valiosas— que los 4 LLMs rechazan por unanimidad. (3) **Lo único con 100 % medido** (LLM unánime + pawsx en SÍ, 13/13) tiene n demasiado pequeño: IC95 % de Wilson [0.77, 1.00]. Ninguna regla supera el umbral de adopción del 97 %.

9.4. Decisión: el panel se degrada a triaje asistente

El resultado negativo es informativo (como E-4 en s8) y *define* la metodología final: el panel no etiqueta — **ordena y pre-vota**. El script `06_t1_panel_triage.py` aplica los 6 votantes a los 120 candidatos minados, los ordena (SÍ-fuertes → desacuerdos → NO-fuertes) y genera un **visor HTML de verificación rápida** (`t1_visor_verificacion.html`): par lado a lado, votos del panel a la vista, teclas S/N/D, progreso persistente y exportación a JSON. La verificación humana baja de ~ 2 h a ~ 10 –15 min (~ 5 s/par con el pre-voto como guía), y el **test_v5 resultante queda 100 % verificado por humano** — provenance limpio, sin etiquetas *silver*, coherente con la regla vigente desde s7 (“el test debe ser humano verificado y sellado”). El sellado lo cierra `07_t1_sellar_test_v5.py`: `test_v5.json = test_v4 íntegro` (109 pares, intacto como benchmark canónico histórico) + los nuevos verificados (solo decisiones SÍ/NO; los “dudoso” se excluyen), con provenance por par (**fuentes**, **verificacion**, sugerencia del panel conservada para trazabilidad), anti-duplicado contra `test_v4` y re-chequeo de fuga contra todos los conjuntos de entrenamiento históricos. La conclusión citable: *la ampliación de tests de paráfrasis académica no se puede automatizar con exactitud ≥ 97 % mediante paneles LLM+especialistas; el humano sigue siendo necesario, pero el triaje del panel reduce su esfuerzo ~ 90 %*.

9.5. Cierre de T1: verificación humana, test_v5 sellado y evaluación

La verificación de los 120 candidatos en el visor tomó una sesión corta (objetivo del triaje cumplido): **19 SÍ · 98 NO · 3 dudosos**. El cruce panel–humano validó el diseño *en campo*:

Pre-voto del panel	<i>n</i>	Humano SÍ	Humano NO	Duda
SÍ (unánime LLM + especialista)	12	7	4	1
SÍ (unánime LLM)	8	3	4	1
SÍ (mayoría)	9	8	0	1
SÍ (especialistas)	1	1	0	0
DESACUERDO	30	0	30	0
NO (mayoría / unánime)	60	0	60	0

Dos lecturas: **(a)** como *ordenador* el triaje fue perfecto — los 19 positivos salieron todos de las categorías SÍ-inclinadas (cero positivos escondidos en los desacuerdos y los NO); **(b)** la precisión del “SÍ fuerte” *en campo* fue 58 % (7/12) frente al 100 % (13/13) medido sobre el ground truth — la confirmación empírica definitiva de que adoptar el panel como etiquetador autónomo habría contaminado el test con ~ 40 % de falsos positivos. La decisión metodológica de la §9 queda doblemente respaldada.

test_v5 sellado. 225 pares = `test_v4` íntegro (109) + 116 nuevos (18 SÍ / 98 NO; 1 duplicado y 3 dudosos excluidos; **0 fugas** en el re-chequeo contra todos los trains). Composición final: **75 SÍ / 150 NO — 33.3 % de positivos**, mucho más cercano a la prevalencia realista estimada en s6 (~ 24 %) que el 52.3 % de `test_v4`. Es, por diseño, **el benchmark más duro del proyecto**: sus 98 negativos nuevos provienen de las bandas más altas de similitud (rechazados por el humano) — adversarios por construcción.

Evaluación de los modelos de producción (umbral 0.5, probabilidad calibrada; `08_evaluar_test_v5.py`):

Modelo	test_v5 (225)	subset test_v4 (109)	subset nuevos (116)
	F1 / AUROC	F1 / AUROC	F1 / AUROC
XLM-R v4 (desplegado)	0.677 / 0.810	0.781 / 0.813	0.493 / 0.859
BETO v3	0.714 / 0.836	0.820 / 0.828	0.507 / 0.907
Ensamble soft	0.670 / 0.837	0.769 / 0.829	0.493 / 0.911

Nota de sanidad: el subset test_v4 reproduce al checkpoint desplegado (0.781/0.813 aquí vs 0.791/0.819 medido en T2; la diferencia es `max_length` 512 vs 256). El “0.812/0.839” histórico de E-5 es la media de 3 *re-entrenamientos*, no el checkpoint desplegado — distinción que este informe deja explícita.

Lectura honesta del test_v5 (y el “triple número”)

(1) **La caída del F1 es de diseño, no una regresión:** con 33 % de prevalencia el mismo clasificador genera más FP por cada TP posible y el F1 baja mecánicamente; el **AUROC — insensible a prevalencia — se mantiene en ~ 0.81 – 0.84** , igual que en test_v4: el *ranking* del modelo no empeoró, el test se volvió más realista. (2) En los 116 nuevos el recall es perfecto (FN=0 en los tres modelos) pero la precisión cae a ~ 0.33 : los negativos minados de alta similitud cuantifican con precisión el problema de FP diagnosticado en E-4 (s8). (3) **BETO v3 supera a XLM-R en test_v5** (0.714 vs 0.677) y el ensamble da el mejor AUROC (0.911 en los nuevos) — refuerza al ensamble como configuración robusta. (4) El encuadre pasa de doble a **triple número: test_v3_limpio** (fácil-histórico) · test_v4 (robusto-balanceado) · **test_v5 (robusto-realista, el más duro)** — cada cifra siempre con su etiqueta. (5) Mejora candidata anotada para v6: re-elegir umbral/temperatura para prevalencia realista (el 0.5 sobre-dispara con 33 % de positivos).

10. Cierre: las cuatro mejoras pendientes implementadas en el sistema v6

Con la validación del asesor para continuar el trabajo (el despliegue queda a la espera de su indicación), los cuatro pendientes anotados en las secciones anteriores se **implementaron y verificaron** en `semana_9/sistema_v6/`. Todo respaldado en la bitácora (Bloques R–U).

10.1. Mejora 3 · Umbral operativo para prevalencia realista

Experimento `11_recalibrar_umbral_test_v5.py` con la *ruta de inferencia de producción* (`core_v4._Modelo: backend int8, T=4.35`): CV 5-fold estratificado dentro de test_v5 (θ elegido en los folds de ajuste, F1 medido en el fold restante — nadie elige θ donde se mide) más θ operativo global declarado.

Modelo	F1@0.5 (FP)	θ^*	F1@ θ^* (FP)	CV insesgado	θ estable
XLM-R int8	0.663 (54)	0.67	0.694 (38)	0.677 $\pm$ 0.061	sí [0.67–0.69]
BETO v3	0.714 (60)	0.51 $\approx$ 0.5	0.715	0.688 $\pm$ 0.079	sí, sin ganancia
Ensamble	0.667 (55)	0.35	0.695	0.665 $\pm$ 0.053	no [0.33–0.58]

Decisión: solo XLM-R adopta $\theta=0.67$ (FP –30 %, precisión 0.54→0.61; el CV confirma que generaliza). El umbral vive en la `calibracion.json` del backend donde fue *medido* (diseño “umbral por backend”: `_Modelo` lo lee con fallback a 0.5) y la zona de duda del árbitro LLM pasa de la franja fija [0.35, 0.65] a $\theta \pm 0.15$ (seguir a la frontera real de decisión). BETO queda en 0.5 (ya óptimo) y el ensamble no se toca (θ^* inestable entre folds = ganancia espuria).

10.2. Mejora 2 · “Modo 2 federado” integrado (R9.2)

El prototipo de E-I2 pasó a módulo de producción `federado.py` (carga perezosa de MarianMT `es→en + beto-pawssx`; `FEDERADO=0` lo desactiva; sin red degrada con mensaje claro), endpoint `POST /api/buscar_federado` y un checkbox “*también corpus científico externo (OpenAlex)*” en el Modo 2 de la interfaz, con tarjeta de resultados (traducción interna visible, prob de solapamiento por obra, enlace a OpenAlex y la nota honesta: título+abstract, no texto completo). Verificado de punta a punta con red viva; ejemplo curado en la pestaña Ejemplos.

10.3. Mejora 1 · Capa de oraciones ACTIVA (T4 integrada)

Con los artefactos de T4 descargados de Drive (`corpus_v4_oraciones_hnsw.index` 2.6 GB + `id_map` 169 MB → `semana_7/indices/`), la capa se integró *opt-in* (`ORACIONES=1`): `_cargar_oraciones()` carga el índice y **solo** el arreglo oración→párrafo padre (~16 MB; los textos no se cargan), y la búsqueda fusiona **tres listas** en el RRF. Verificado en vivo: **recuperación: híbrida** (**semántica HNSW + léxica BM25 + oraciones, RRF**); en el reporte de tesis, una oración fina ≥ 0.80 rescata su párrafo padre (el caso “una sola oración copiada dentro de un párrafo original”). **RAM medida:** ~12 GB estables en la máquina de 15 GB; estimado ~10–11 GB en el Space de 16 GB (confirmar al desplegar).

Bug de concurrencia cazado durante la activación

Dos peticiones simultáneas entraron a `_cargar_oraciones()` y cargaron el índice de 2.6 GB **dos veces en paralelo** (pico: 265 MB de RAM libres — en el Space habría tumbado el contenedor). El mismo patrón existía desde `s7/s8` en `_cargar_indice` y `_cargar_bm25`, mitigado solo por el pre-warm. **Fix:** double-checked locking (`threading.Lock`) en los tres cargadores perezosos; verificado con dos búsquedas concurrentes → una sola carga en el log.

10.4. Mejora 4 · Votos pendientes del panel

Los 193 votos pendientes del panel quedaron **completados** (146 recuperados el 3 de julio y los 47 restantes el 4 de julio, con la cuota diaria de Groq renovada; el cache cerró en 2 478 votos). No cambia ningún veredicto: `test_v5` quedó sellado con verificación humana; esto solo completa la trazabilidad del cache del panel.

10.5. Selector de corpus en tesis completa: CBI, externo o ambos

Extensión de la capa federada al análisis de *tesis completa* (antes solo en el Modo 2 fragmento). Un selector “comparar contra” ofrece: **Corpus CBI** (el reporte local por secciones de siempre), **Corpus científico externo** (solo OpenAlex, sin cargar el índice local) y **Ambos** (reporte local + sección de fuentes externas).

Como consultar OpenAlex por cada uno de los cientos de fragmentos de una tesis es inviable (latencia y *rate-limit*), la capa externa **muestrea**: filtra boilerplate y toma una muestra distribuida de ~8 fragmentos de contenido, los traduce, consulta la API y **agrega las obras por identificador** (cuántos fragmentos coinciden y su mejor probabilidad). La cobertura se **declara** en la salida (“muestra de *N* fragmentos, no exhaustiva”; título+abstract, no texto completo) y el % de similitud formal sigue siendo contra el corpus CBI. Implementado en `federado.analizar_texto`, `app_unificada._correr_analisis(corpus=)` (parámetro en `/api/analizar` y `/api/analizar_doc`)

y el selector en la interfaz; verificado de extremo a extremo (una consulta real devolvió obras de *Particology* y *Chemical Engineering Science* con probabilidad de solapamiento baja = afinidad de tema, no paráfrasis).

10.6. Cinco brechas de construcción corregidas + interfaz “B+”

Del análisis retrospectivo (“qué se nos pasó por implementar”) se corrigieron: **(1)** zona de duda del árbitro alineada al umbral del backend; **(2)** el cross-encoder del Modo 2 fragmento pasa a **prob_batch** (un forward por lote, $\sim 2\text{--}3\times$ más rápido que el par-a-par con hilos); **(3)** híbrido completo en el reporte de tesis (banda BM25 ampliada + rescate por oraciones); **(4)** % de **palabras idénticas** por coincidencia (estilo Turnitin, **diffli**) en Modo 1, candidatos y evidencia; **(5)** **citas entrecomilladas** detectadas y excluidas del % relevante (como el boilerplate; categoría propia en la dona, el filtro y la línea de tiempo). La interfaz evolucionó en la dirección “B+” elegida por el alumno: *badge* de estado en vivo (backend $\cdot \theta \cdot$ capas cargadas, vía **GET /api/estado**), historial de análisis recientes (localStorage), chips nuevos (θ , idéntico %), micro-interacciones y foco accesible. Validación: **node -check** del JS, **py_compile**, y render verificado con captura de navegador headless.

11. Auditoría crítica del sistema y dos correcciones de fondo

Tras integrar las mejoras se ejecutó una **batería funcional de caja negra** (**12_plan_pruebas_v6.py**, aserciones sobre el servidor en vivo: dos modos, tres clasificadores, seguridad, robustez). La corrida original ejecutó **42 aserciones (42/42 pasan)**; tras las dos correcciones de esta sección la suite se re-ejecutó *omitiendo las 8 pruebas del federado que requieren red* (opción **--skip-federado**), y ese es el respaldo vigente en disco: **35/35 pasan** (**12_plan_pruebas_v6.json**, 4 jul). La cobertura del federado quedó re-verificada al día siguiente en la suite exhaustiva de la §16.2 (grupo F, 4/4). Pero una auditoría crítica de los *valores* (no solo de los campos) destapó dos problemas serios que las pruebas estructurales no capturan — porque prueban que la salida existe, no que sea correcta.

KC-1 · El clasificador por defecto no reconocía paráfrasis obvias

Sobre cinco paráfrasis de manual, el default (XLM-R) las clasificó **0/5** como paráfrasis; el ensamble soft-voting también **0/5** (el promedio ahoga los aciertos del otro modelo); **BETO 5/5**. Diagnóstico: XLM-R (fp32 e int8) es un modelo *precisión-primer* de recall bajo — para el par cobre+ácido nítrico da logits [2.91, −3.44], probabilidad cruda 0.002. **Corrección:** el clasificador por defecto pasó a **BETO v3** (recall 1.0); XLM-R queda seleccionable como opción rápida (int8) y multilingüe. La copia literal real se sigue detectando.

KC-2 · El Similarity Score medía tema, no plagio

El número titular contaba fragmentos con similitud semántica ≥ 0.80 ; un texto académico **100 % original** sobre un tema presente en el corpus daba **100 % de similitud**, contradiciendo la tesis del proyecto (“similitud $\neq$ plagio”). **Corrección:** el titular pasó a contar solo coincidencias **confirmadas** por el clasificador (copia/paráfrasis); la similitud alta sin confirmar va a un contador “por revisar” aparte, y el bruto ($\text{sim} \geq 0.80$) queda como cifra informativa. El texto original bajó de **100 % a 20 %** (el residual es la precisión del clasificador, ahora mostrada con honestidad). Es la diferencia entre un “% de texto coincidente” (lo que hace Turnitin) y una similitud de tema disfrazada de plagio.

Lección metodológica (coherente con el estilo del proyecto): *pruebas verdes no son salida correcta*. Las 42 aserciones estructurales de la corrida original pasaban mientras el sistema simultáneamente ignoraba paráfrasis evidentes y acusaba de plagio a texto original. Auditar los *números*, no solo los campos, es lo que convirtió un demo en un sistema defendible.

12. KC-3 y KC-4: auditoría de las métricas del reporte de tesis con PDFs reales

Probando el sistema con **tesis reales del corpus** (carpeta `documentos_para_probar_v6`) surgieron dos observaciones sobre las métricas del reporte de tesis completa. El caso testigo es `UAM1642.pdf`, una tesis que *está* en el corpus (autocopia): el banner “posible copia” se dispara correcto (cobertura 84.6 %, similitud mediana 0.92), el *timeline* sale todo rojo y la fuente principal aparece a similitud 100 % — todos los indicadores de “mismo documento” funcionan — pero el *Similarity Score* titular marca ~65–77 %, no ~100 %. La pregunta legítima: *¿por qué, si es el mismo documento?*

12.1. Cómo se calcula el Similarity Score (y por qué ~77 % es correcto)

El titular es $\text{similitud_relevante_pct} = 100 \times n_{\text{confirmadas}} / n_{\text{fragmentos}}$, donde $n_{\text{confirmadas}}$ cuenta los fragmentos cuyo mejor match se clasifica como **copia o paráfrasis** (severidad ≥ 2), *excluyendo* boilerplate y citas (fix KC-2). Para `UAM1642` (505 fragmentos de contenido): 324 confirmadas $\rightarrow$ 64.2 %. La brecha frente al 100 % tiene dos componentes, y **uno de ellos es correcto por diseño**:

1. **Boilerplate excluido (correcto)**. 71 fragmentos (~14 % del documento) son portadas/plantillas institucionales, iguales en *todas* las tesis de la UAM; el fix KC-2 los excluye del titular porque no son plagio. Una autocopia por tanto **nunca puede ni debe llegar al 100 %**: su techo honesto es $(n_{\text{sosp}} - n_{\text{boiler}} - n_{\text{citas}}) / n = 411 / 505 = 81.4$ %.
2. **Sub-conteo por ruido de extracción (KC-3, sí era mejorable)**. Ver abajo.

KC-3 · Copias con ruido de extracción caían a “verificar”

Al subir un PDF, el texto re-extraído queda **desalineado** respecto al indexado (misma ventana 400/*stride* 200, pero posiciones de carácter distintas por la extracción). En los fragmentos con tablas, fórmulas o caracteres OCR-ruidosos, el % de palabras idénticas y el BLEU-2 caen *justo por debajo* del umbral de copia (la regla exigía $\text{sim} \geq 0.85 \wedge \text{pct_id} \geq 55$). Calibrado sobre los fragmentos “verificar” reales de `UAM1642`: tienen **sim 0.90–0.94** (altísima — son el mismo documento) pero pct_id 35–55, mientras las copias confirmadas tienen $\text{pct_id} \geq 45$ (mediana 75). El resultado: copias verbatim demotadas a “alta similitud — verificar” (severidad 1), que no cuenta en el titular.

Corrección KC-3 (en `core_v4.reporte_tesis_completa`): (i) se añadió a `_tipo_por_similitud` una regla robusta al desfase — $\text{sim} \geq 0.90 \wedge \text{pct_id} \geq 35 \rightarrow \text{copia casi literal}$ (severidad 3); y (ii) en `_tipo_sev`, la **evidencia léxica** de copia dura (severidad 3) *gana sobre el cross-encoder*: una copia verbatim es copia por hecho léxico (texto idéntico), independiente de lo que juzgue el clasificador de paráfrasis *semántica*. El corte se eligió con margen de seguridad: la afinidad de tema entre tesis *distintas* vive en $\text{sim} < 0.90$ con $\text{pct_id} < 20$, así que no puede activar la regla.

Verificación en vivo (servidor real, misma extracción antes/después — el único cambio es el código):

Documento	Confirmadas (antes→después)	Titular	Falsos positivos
UAM1642 (autocopia)	324 → 331	64.2 % → 65.5 %	—
arXiv EN (no es copia del corpus)	0 → 0	0.0 % → 0.0 %	0 ✓

La mejora es **deliberadamente conservadora**: solo rescató las 7 copias inequívocas de $\text{sim} \geq 0.90$; los ~ 80 fragmentos restantes “por revisar” están en $\text{sim} 0.80\text{--}0.90$ con solape léxico moderado y *legítimamente* siguen marcados para revisión humana (a esa similitud, con ese solape, son genuinamente ambiguos). La prueba de seguridad sobre el arXiv en inglés (documento que *no* está en el corpus) confirma **cero falsos positivos**: sus fragmentos afines de tema ($\text{sim} 0.81\text{--}0.85$) no cruzan el umbral. **Conclusión honesta**: el $\sim 65\text{--}77\%$ de una autocopia **no era un error** — es la medición correcta de “copia de contenido confirmada, sin boilerplate”; el veredicto “mismo documento” se comunica por el banner, el *timeline* y la fuente al 100 %, no por el titular.

KC-4 · El donut “Tipos de coincidencia” no reconciliaba con el titular

El gráfico de dona se calculaba sobre las **top-350 coincidencias por similitud** (evidencia), no sobre el documento completo. Para UAM1642 mostraba *boilerplate*=37 y *verificar*=18, mientras el titular global reportaba $n_{\text{boilerplate}} = 71$ y $n_{\text{por revisar}} = 87$: el pastel y los números grandes **contaban poblaciones distintas** (una muestra sesgada a alta similitud vs. el documento entero).

Corrección KC-4: el donut ahora usa `tipos_full`, la distribución de tipos sobre **todos** los sospechosos del documento (el mismo conjunto y la misma lógica `_tipo_sev` con que se cuentan las confirmadas/por-revisar del titular). Verificado en vivo: para UAM1642 el donut suma $484 = n_{\text{sospechosos}}$ con *boilerplate*=71 = $n_{\text{boilerplate}}$ — reconcilia exacto; para el arXiv, $8 = 8$.

Lección (misma familia que KC-1/KC-2): las dos correcciones salieron de *probar con documentos reales del dominio* y auditar que cada número mostrado sea coherente con los demás. KC-3 afinó la exactitud (rescate de copias sin falsos positivos); KC-4 restauró la consistencia interna del reporte. Ambas verificadas de extremo a extremo contra el servidor en vivo.

13. Resumen ejecutivo profundo: ablación de LLM y rediseño del prompt

El reporte de tesis incluye un *resumen ejecutivo* generado por un LLM. Al probarlo con tesis reales se detectaron tres límites: (1) el modelo **nunca veía el texto** de la tesis — solo recibía porcentajes y conteos —, así que no podía resumir de qué trata el documento; (2) no usaba la señal de *autocopia* (`documento_original`) ni distinguía copia confirmada de similitud de tema; y (3) no explicaba las fuentes externas de OpenAlex. Además el *prompt* mezclaba el % confirmado con el conteo bruto.

13.1. Ablación de modelos (empírica, estilo del proyecto)

Con el nuevo *prompt* (que incluye un extracto real del documento y las métricas correctas), se compararon cuatro LLMs disponibles vía Groq sobre el reporte real de UAM1642 (script `16_ablacion_resumen_llm.py`, respaldo `16_ablacion_resumen_llm.json`). Gemini quedó fuera (SDK deprecado y cuota agotada desde s4).

Modelo (Groq)	Latencia	Tokens	Calidad del resumen
gpt-oss-120b	1.4 s	499	Ganador: tema preciso (capta que la tiourea es alternativa menos tóxica al cianuro), métricas exactas, mejor encuadre honesto (“reutilización de material ya publicado”)
llama-4-scout-17b	1.2 s	304	Muy bueno y el más rápido; correcto y honesto, algo menos rico
llama-3.3-70b (actual)	1.8 s	402	Buen tema, pero error de métrica: leyó “87 frag” como “87 %”
qwen3-32b	1.5 s	500	Razona en <think> y agota el presupuesto → salida incompleta

Decisión: gpt-oss-120b para el resumen ejecutivo

El modelo del resumen pasó de Llama 3.3 70B a **gpt-oss-120b** (el modelo abierto de OpenAI servido por Groq), que dio las métricas más exactas y la mejor comprensión del tema. El árbitro LLM del Modo 1 y la explicación de pares siguen con Llama 3.3 70B (no se tocan). Configurable por entorno (GROQ_MODEL_RESUMEN).

13.2. Rediseño del prompt y verificación en vivo

El *prompt* nuevo entrega al modelo: un **extracto de contenido** del documento (saltando la portada), las **métricas correctas** (copia confirmada como titular, “por revisar” y boilerplate por separado), la **señal de autocopia** y, cuando existen, las **obras externas** de OpenAlex. La salida tiene estructura fija: *de qué trata el documento · veredicto de similitud · (fuentes externas) · recomendación*. El contraste es directo:

Antes (genérico)	Ahora (gpt-oss-120b, medido en vivo)
“similitud global del 77.19 %... se recomienda revisar las secciones con mayor similitud” — sin tema, sin distinguir autocopia.	De qué trata: diseño de una planta para obtener oro y plata de un concentrado mineral, con lixiviación y extracción por solventes, sustituyendo el cianuro por tiourea... Veredicto: 64.16 % (324 frag.) de copia confirmada, 87 para revisión humana, coincidencia 84.6 % con un trabajo previo (posible mismo documento); 71 frag. de boilerplate... no necesariamente plagio intencional.”

Verificado en vivo contra el servidor: el resumen identifica correctamente el tema, reporta las métricas exactas, detecta la autocopia y — con un *payload* de fuentes externas — añade la sección que las explica (“afines de tema, no paráfrasis”). Cambios en `app_unificada.py` (endpoint `/api/resumen_ejecutivo`, `_groq` con modelo configurable) y `web/unificado.html` (envía el extracto de texto, la señal de autocopia y las obras externas). `py_compile` y validación de JS correctas.

14. Soporte bilingüe: analizar documentos en inglés (interno y externo)

El corpus CBI es español, pero un alumno puede subir una tesis o un artículo en inglés y querer compararlo tanto contra el corpus interno como contra la literatura externa. La solución se apoya directamente en el hallazgo de **T3** (§6): la verificación *cross-lingüe* es↔en del cross-encoder **colapsa**

(F1 0.066), mientras es-es y en-en funcionan. Por tanto, la regla es **traducir para igualar el idioma antes de verificar**, nunca comparar a través de idiomas.

14.1. Arquitectura por superficie

Superficie	Regla bilingüe
Modo 1 (comparar dos textos)	Se detecta el idioma de cada texto; el que esté en inglés se traduce en→es y se verifica es-es.
Modo 2 fragmento vs CBI	Fragmento en inglés → traducir en→es → pipeline es-es normal (la recuperación MiniLM ya es multilingüe, pero verificación y léxico BM25/BLEU necesitan mismo idioma).
Modo 2 tesis completa vs CBI	El documento en inglés se traduce en→es <i>antes</i> de fragmentar; todo el reporte (secciones, <i>spans</i> , evidencia) corre sobre la traducción, declarada al usuario.
Federado (OpenAlex, inglés)	Al revés: si el documento ya está en inglés se consulta directo (verificación en-en), <i>saltando</i> la traducción es→en que sí necesita el flujo en español.

Módulo nuevo `idiomas.py`: detección con `langdetect` (sobre una muestra central, para evitar portadas ruidosas) y un traductor MarianMT es↔en por lotes (*greedy*), con carga perezosa y caché por texto. Los dos modelos (`opus-mt-en-es` y `opus-mt-es-en`) ya estaban en caché local.

14.2. Costo y optimización: traducir solo los sospechosos

La primera versión traducía el *documento completo* antes de fragmentar (~ 1.63 s/fragmento en CPU); una prueba real del alumno con un artículo de 107 680 caracteres tardó **994 s** (~ 16.5 min) — inaceptable. La causa: se traducían los ~ 500 fragmentos aunque casi ninguno tuviera coincidencia.

Optimización: recuperación cross-lingüe + traducir solo lo necesario

Como el recuperador MiniLM es **multilingüe**, la búsqueda funciona sin traducir. Solo la *verificación* (cross-encoder) necesita mismo idioma. Por tanto se reescribió el reporte para: (1) fragmentar y buscar sobre el texto en inglés (cross-lingüe); (2) traducir en→es **únicamente los fragmentos sospechosos** (los que recuperaron candidato) antes de la verificación es-es. Para un artículo que no está en el corpus español, eso son ~ 8 traducciones en vez de ~ 500 .

Resultado medido en vivo: el mismo artículo de 107 680 caracteres bajó de **994 s a 189 s** ($5.3\times$), con el mismo veredicto correcto (0 coincidencias confirmadas: no está en el corpus). El inglés pasó de $\sim 8\times$ más lento que un documento equivalente en español a $\sim 1.3\times$: los 189 s son mayormente los *embeddings* de los 538 fragmentos (~ 75 s, inherentes a cualquier análisis), no la traducción. El resaltado muestra ahora el texto **original** en inglés; las tarjetas de evidencia muestran la traducción (para que sean comparables con la fuente en español).

14.3. Verificación en vivo

Las cuatro superficies se probaron contra el servidor real (`17_verificacion_bilingue.json`):

Prueba	Resultado (medido)
Modo 1 · par EN vs ES (mismo contenido)	Idiomas en/es detectados, A traducido en→es; es_parafrasis=sí , “copia casi literal”, prob 0.815, sim 0.985 — antes de esto el par cruzado <i>colapsaba</i> (T3).
Modo 2 · fragmento EN vs CBI	Traducido a español, recuperación híbrida (HNSW+BM25+oraciones); mejor match UAM0678.PDF , sim 0.86, “paráfrasis”.
Federado · fragmento EN	Usó el inglés directo (sin re-traducir), 4 obras de OpenAlex (<i>hydrometallurgical recovery...</i>), prob máx 0.0095 = afinidad de tema.
Modo 2 · tesis EN (12k car., truncada)	Idioma en detectado, documento traducido en→es, 50 fragmentos, titular 0 % en 175 s — correcto: un artículo de arXiv sobre datos sintéticos no está en el corpus CBI.

Limitaciones declaradas: la traducción automática (MarianMT) añade ruido, así que el la verificación de un documento en inglés corre sobre la traducción de sus fragmentos sospechosos, no sobre el original (se avisa en la interfaz con “*Documento en inglés*”). Con la optimización de traducir-solo-sospechosos (§anterior) el reporte de una tesis en inglés completa bajó de ~16.5 min a ~3 min en CPU. Cambios en `idiomas.py` (nuevo), `core_v4.py` (Modo 1, fragmento y reporte), `federado.py` (inglés directo) y `web/unificado.html` (avisos de idioma). `py_compile` y validación de JS correctas.

14.4. Resultados del corpus externo enriquecidos con % de similitud y métricas

A petición del alumno, cada obra del corpus externo (OpenAlex) muestra ahora, además de la probabilidad de solapamiento (paráfrasis contra el *abstract*, vía beto-pawxs), una **similitud semántica de tema** (coseno de MiniLM entre la consulta y el *abstract*, en %), con una barra de calor; y la sección trae un panel de **métricas agregadas** — similitud máxima/media, solapamiento máximo, número de obras y cuántas superan el umbral — que funciona como *score* del corpus externo. Verificado en vivo: consulta sobre generación de datos sintéticos → “*A Survey of Large Language Models*” con similitud de tema **57.7 %** y solapamiento **1.3 %** (afinidad temática sin paráfrasis, la respuesta honesta). Cambios en `federado.py` (MiniLM propio + **sim** por obra + métricas) y `web/unificado.html` (chips de % y panel de métricas en las dos tarjetas federadas).

15. Rediseño de la interfaz: dirección visual “Consola analista”

La interfaz heredada, aunque funcional, se parecía demasiado a la del sistema v5. Se abordó un rediseño con método de estudio de diseño: se propusieron **tres direcciones visuales** —*Editorial académico* (serif escolar, retícula suiza), *Consola analista* (oscuro, vidrio, acento eléctrico, dispersión 3D) y *Bento data-forward* (retícula modular clara)— presentadas como *mockups* interactivos aplicados a datos reales de una tesis. El alumno eligió la **Consola analista**, con el requisito de implementarla en **modo claro y oscuro** (no solo oscuro).

15.1. Sistema de diseño (ambos temas)

El cambio es puramente de presentación (CSS y un poco de JS de dibujo); el *backend* y la lógica del cliente no se tocan (los nombres de clase se conservan). Se reescribió la capa de *tokens* para los dos temas: paleta eléctrica azul→violeta sobre neutros fríos, más *tokens* nuevos de **-glow**, **-glass** y **-blur**. Sobre eso: fondo con **gradientes ambientales** radiales, **vidrio** en tarjetas, barra superior y lateral (*color-mix* translúcido + *backdrop-blur* + realce interior), y acentos con resplandor (barra

de progreso, hover de evidencia, números de métricas con degradado). El tema oscuro es el mundo canónico (carbón azulado); el claro es su versión de día, con el mismo acento y un glow más sutil.

15.2. Piezas de datos como parte del sistema

- **Gauge del *Similarity Score***: un arco SVG con el color de severidad del score (verde/ámbar/rojo) y *glow*, con el porcentaje al centro — reemplaza el anillo plano anterior.
- **Dispersión 3D con densidad kernel** (*canvas*): cada coincidencia es un punto en similitud $\times$ probabilidad $\times$ solape léxico (profundidad), coloreado por severidad, con halos de densidad y rotación suave que respeta `prefers-reduced-motion`. Es la materialización de los “gráficos de solapamiento / dispersión 3D” solicitados, y sustituye al diagrama 2D estático.
- **Donut, histograma y línea de tiempo** recibieron el mismo lenguaje: el donut con anillo de fondo, segmentos separados con extremos redondeados, *glow* y total al centro; el histograma con barras redondeadas, resplandor y texto sensible al tema; la línea de tiempo como riel hundido con brillo superior y cada franja con el *glow* de su propio color al pasar el cursor.
- **Tiles KPI tipo bento** en el *Similarity Score*: una rejilla de cuatro tarjetas (confirmadas, por revisar, boilerplate, fuentes) con número tabular grande y etiqueta monoespaciada. El *gauge* es ampliable: al pulsarlo se abre en un modal a mayor escala con el desglose KPI y la explicación de la métrica; la dispersión 3D también se abre ampliada e interactiva en el modal (cada *canvas* gestiona su propia animación).

Verificación (*headless* Chrome): la interfaz renderiza limpia en **claro y oscuro** (capturas `ui_claro.png` / `ui_oscuro.png`); el *gauge* y la dispersión 3D se probaron con **datos de evidencia reales** de una tesis en ambos temas (`charts_claro.png` / `charts_oscuro.png`), con los colores de severidad resolviéndose correctamente. El JS del cliente valida (`node --check`) y el `<style>` queda balanceado. Cambios acotados a `web/unificado.html`.

Verificación de extremo a extremo. Para confirmar el reporte *completo* (no solo las piezas por separado), se inyectó el reporte **real** de la tesis UAM1642 —generado por el *backend* vigente— en el *frontend* real y se renderizó *headless* en ambos temas (`rep_claro.png` / `rep_oscuro.png`): todos los componentes aparecen correctos con datos reales —banner de autocopia, *flags*, línea de tiempo, documento resaltado, *gauge* (65.54 %), tiles KPI (331/80/71/8), donut, métricas de triple número y dispersión 3D—. En esa pasada se detectó y corrigió un rótulo desactualizado: el botón del resumen ejecutivo decía “(Llama 70B)” cuando el modelo del resumen ya es gpt-oss-120b; se cambió a “Generar resumen con IA” (el modelo real se muestra tras generar). El árbitro del Modo 1 sí sigue siendo Llama 3.3 70B, sin cambios.

15.3. Rediseño de la salida de resultados (claridad y comprensión)

Probando con documentos reales, se rediseñó la presentación del reporte de tesis completa para que los resultados se entiendan mejor: (1) se **eliminó el campo “título (opcional)”** del formulario (sin utilidad); (2) el **resumen ejecutivo se genera automáticamente** al abrir el reporte (antes requería pulsar un botón); (3) el *Similarity Score* pasó a un **bloque protagonista** con un *badge* de veredicto (“Riesgo bajo / moderado / alto”), tiles KPI, y —a petición del alumno— una explicación explícita de **qué mide el número**: “el % de fragmentos con copia o paráfrasis *confirmada* contra el corpus, no similitud de tema; la afinidad temática sin confirmar y el boilerplate se cuentan aparte”; y (4) las coincidencias se separaron en **secciones rotuladas y distintas**: “*Coincidencias en TESIUAMI*” (corpus interno), “*Coincidencias en literatura científica externa*” (OpenAlex, solo si se consultó el corpus externo) y “*Análisis técnico*” (modelo, dispersión 3D y evidencia). Además, cada

obra externa muestra su % de similitud de tema y el panel de métricas del corpus externo. Verificado con capturas *headless* en ambos temas.

16. Verificación exhaustiva del sistema v6 y viabilidad de una arquitectura RAG

Cierre de la semana (Bloque Y de la bitácora): una verificación *de extremo a extremo* de que **todo** lo implementado en la semana 9 está ejecutándose correctamente en el sistema v6, más un experimento que responde con datos una pregunta de arquitectura abierta: ¿es viable usar RAG (*Retrieval-Augmented Generation*) en el sistema?

16.1. Metodología: pruebas seguras en memoria

La sesión previa de pruebas terminó congelando la máquina local: el servidor (~9 GB con modelos e índices) y un script que cargaba los modelos *por segunda vez* agotaron los 15 GB de RAM. Se adoptó una receta operativa que se aplicó durante toda la verificación, con **cero congelamientos**:

1. **Tope de memoria por cgroup**: el servidor corre dentro de `systemd-run --user --scope -p MemoryMax=9G..13G` — si la carga excede el tope, el kernel termina *solo al servidor*, nunca al escritorio.
2. **Toda prueba va por HTTP** contra el servidor vivo (los scripts de prueba *no* cargan modelos; los modelos viven una sola vez, en el servidor).
3. **Monitor de RAM en paralelo** con umbral de alerta, durante toda corrida.

Con el stack completo cargado (índice HNSW + BM25 + **capa de oraciones** + federado + tres clasificadores con int8) la RAM del servidor midió **11.7 GiB (pico 11.9)** — dato directo para el despliegue: **cabe en el Space de 16 GB con ~4 GB de margen**. Sin la capa de oraciones el stack usa ~8.6–9 GiB.

16.2. Suite exhaustiva: 33/33 aserciones pasan

El script `14_pruebas_s9_exhaustivas.py` (respaldo `14_pruebas_s9.json`) verifica en vivo cada mejora de la semana, organizada en siete grupos:

Grupo	Qué verifica (mejora de s9)	Resultado
A	<code>/api/estado</code> · backend onnx-int8 ($T=4.35$) · umbral por backend ($\theta=0.67/0.5$) · capa de oraciones activa (ORACIONES=1)	5/5
B	KC-1: default BETO · 5/5 paráfrasis de manual · copia casi literal · no-relación	6/6
C	<code>pct_palabras_identicas</code> (94 % casi-copia vs. 27 % paráfrasis) · zona de duda = <code>clf.umbral ± 0.15</code>	4/4
D	Modo 2: recuperación híbrida de 3 capas (HNSW+BM25+oraciones, RRF) · <code>pct_identico</code> en candidatos · 2 búsquedas concurrentes (lock)	5/5
E	Reporte: campos KC-2 · texto original → titular 0 % (bruto 70.6 % aparte) · cita entrecomillada detectada · copia verbatim confirmada · telemetría	6/6
F	Federado: traducción interna + candidatos OpenAlex + nota honesta · selector <code>corpus=externo</code> en tesis completa	4/4
G	Robustez: entradas vacías/cortas/malformadas → error controlado	3/3
Total		33/33

Dos lecciones de construcción de la propia suite: (1) hay que **calentar las cargas perezosas antes de medir** — la corrida en frío daba falsos negativos de estado (clasificadores sin listar, BM25 apagado) y 92 s de primera búsqueda vs. 3.9 s en caliente; (2) la prueba de citas debe entrecomillar un fragmento *real del corpus*: la exclusión de citas (brecha #5) solo aplica sobre fragmentos con similitud alta, que es exactamente su diseño (cita correctamente atribuida de material existente).

Bug real cazado por la suite: JSON malformado devolvía 500

El `except Exception` genérico del servidor trataba un cuerpo JSON malformado (error del *cliente*) como error del *servidor* (HTTP 500). Corrección quirúrgica en `app_unificada.py`: rama `except (json.JSONDecodeError, ValueError) → 400` con mensaje claro (cubre también parámetros no numéricos). Verificado en vivo (aserción G3). Se anota sin cambiar: comparar con textos vacíos responde `200 + {".error": ...}` — controlado, el frontend lee el campo de error.

16.3. ¿Es viable una arquitectura RAG en el sistema? Medición sobre `test_v5`

El proyecto ya había medido variantes RAG en semanas previas, siempre con resultado negativo: RAG *naive* ($\Delta F1 = -0.301$, s4), RAG estratificado con BETO (F1 0.146, s4), BETO+RAG vs. nativo (0.878 vs. 0.990, s5) y RAG-KB de ejemplos gold (0.700 < 0.842, s6). Lo que faltaba era re-medir la variante más defendible —**RAG-ICL**: recuperar los ejemplos etiquetados más similares del gold y dárselos como *few-shot* a un LLM juez— con los instrumentos actuales del proyecto: el benchmark más duro (`test_v5`) y el mismo recuperador del sistema (MiniLM multilingüe).

Diseño (`15_viabilidad_rag.py → 15_viabilidad_rag.json`): muestra estratificada de 60 pares de `test_v5` (20 SÍ / 40 NO — la prevalencia realista del benchmark, semilla fija), tres brazos, cache reanudable resistente a los 429 de cuota:

Brazo	F1	P	R	Latencia mediana
BETO v3 nativo (producción, <code>/api/comparar</code>)	0.702	0.541	1.000	1.5 s
LLM <i>zero-shot</i> (Llama 3.3 70B, $T=0$)	0.809	0.704	0.950	1.9 s
LLM + RAG-KB (top-3 del gold por MiniLM)	0.714	0.556	1.000	4.7 s

Lecturas honestas:

1. El **RAG-ICL empeora al LLM** ($\Delta = -0.095$ vs. *zero-shot*): los ejemplos recuperados sesgan al juez hacia el SÍ (FP 8 $\rightarrow$ 16, la precisión cae 0.704 $\rightarrow$ 0.556). Es la *cuarta* confirmación del patrón histórico del proyecto: el contexto recuperado no aporta la señal de la tarea, aporta ruido de similitud temática.
2. El LLM *zero-shot* puntúa alto en esta muestra (0.809), pero con $n=60$ el intervalo de confianza es ancho, depende de una API externa (cuota, latencia, no-auditabilidad, no-determinismo operativo) y el panel de la semana 9 ya midió $\kappa \approx 0.52\text{--}0.56$ para LLMs solos sobre 294 etiquetas adversarias (Bloque N). No sustituye al clasificador local; su lugar correcto *ya existe* en el sistema: árbitro opcional en la zona de duda.
3. La parte del patrón RAG que *sí* funciona **ya está integrada**: recuperar-luego-clasificar (Modo 2: HNSW+BM25+oraciones con RRF $\rightarrow$ cross-encoder) y el federado (recuperación en OpenAlex $\rightarrow$ verificación en-en). Lo que se descarta es la *generación* aumentada por recuperación como clasificador del Modo 1.

Veredicto: no adoptar RAG-ICL; el sistema ya implementa la variante de RAG que aporta

Con cuatro mediciones consistentes en cuatro semanas distintas (s4, s5, s6 y s9, esta última sobre el benchmark más duro y con el recuperador de producción), la evidencia es estable: **ninguna variante de RAG supera al clasificador nativo** en el Modo 1, y la variante few-shot además *degrada* al LLM juez. La arquitectura vigente (recuperar-luego-clasificar + árbitro LLM opcional en zona de duda) queda respaldada con datos como la elección correcta.

17. Pruebas con *ground-truth* y dos correcciones de atribución

Para validar que el sistema encuentra *lo que debe*, se crearon **cinco documentos de prueba con contenido conocido**: fragmentos copiados verbatim de tesis CBI reales (cosechados del propio corpus), paráfrasis LLM de esas tesis, boilerplate institucional, homóglifos cirílicos + caracteres invisibles, y *abstracts* verbatim de OpenAlex. Cada documento lleva su *ground-truth* por fragmento (`semana_9/pruebas_ground_truth/`).

Resultados vs. lo esperado: boilerplate correctamente excluido con la copia de contenido detectada; homóglifos cirílicos **cazados por el panel de *flags*** (13 palabras); paráfrasis LLM en “verificar”/“paráfrasis” (recall honesto de BETO); copia externa verbatim identificada por el federado (sim 0.96). El ejercicio destapó dos debilidades de *atribución*, ambas corregidas:

Corrección 1 · La bandera de solapamiento externo ignoraba la similitud alta

Un *abstract* copiado verbatim da similitud ~ 0.96 pero la probabilidad de paráfrasis del cross-encoder se *diluye* al truncar $\rightarrow$ se mostraba como “afín de tema”. Se añadió `UMBRAL_SIM_EXT=0.85`: **posible_solapamiento** se marca también por similitud alta. Verificado: “*Attention Is All You Need*” $\rightarrow$ posible=sí.

Corrección 2 · La copia verbatim podía quedar mal atribuida (fusión de ventana)

Con copias cortas de tesis distintas pegadas, la ventana de fragmentación (400/*stride* 200) mezcla fuentes y la copia real puede quedar rankeada *bajo* una tesis de tema-similar por la similitud semántica (medido: copia `pct_id` 92.6 rankeada #3, debajo de una tesis tema-similar con `pct_id` 18.5). Se corrigió la selección de fuente para **re-rankear el top-K por solape léxico** (`pct_idéntico`) y confirmar copia por léxico independiente de la similitud (una copia con texto idéntico es copia aunque el embedding de una ventana corta dé similitud baja). Resultado en la prueba: de **3 copias (60 %)** a **4 copias (80 %)**, con la fuente verbatim recuperada; sin regresión (el auto-plagio de UAM1642 pasó de 331 a 341 coincidencias confirmadas, sin falsos positivos en los documentos de control).

18. Verificación pre-despliegue: pruebas de funcionamiento y auditoría de seguridad

Antes de ejecutar las acciones de red del despliegue se corrió una verificación integral del sistema v6 con la **configuración exacta del tier de producción** (`INDICE_V4=hns` + `ORACIONES=1` + `ONNX_INT8=1` + federado con red real), con el servidor bajo un *cgroup* de 13 GB (regla anti-OOM de la semana).

Funcionamiento. La suite exhaustiva (script 14) pasó **33/33 aserciones** en 338.8s, cubriendo los siete grupos (estado/int8/umbral por backend, KC-1, métricas nuevas, híbrido de tres capas con concurrencia, reporte KC-2/citas/copia, federado con selector de corpus, y robustez ante entradas inválidas). La RAM del *stack* completo midió **11.5 GiB (pico 11.8)** —consistente con la medición previa y dentro de los 16 GB del Space.

Seguridad. Barrido estático + pruebas en vivo: cero claves en código, JSONs, notebooks y paquete de despliegue; el `.env` real fuera de git (permisos 600); *path traversal* rechazado en `/api/pdf` y `/api/progreso`; cabeceras `nosniff/X-Frame/Referrer` presentes; topes de subida (60 MB) y de cuerpo (4MB) activos; consultas a OpenAlex con `urlencode` y host fijo (sin SSRF). Se aplicaron **dos endurecimientos**:

Endurecimiento 1 · `esc()` escapaba solo `<`

El texto del usuario dentro de atributos `title="..."` (historial “Recientes”) permitía romper el atributo e inyectar un *handler* (self-XSS de severidad baja). La función ahora escapa los cinco caracteres especiales de HTML. Verificado en vivo.

Endurecimiento 2 · *bind* configurable con default seguro

El servidor *bare-metal* escuchaba en 0.0.0.0 (expuesto a la LAN, sin autenticación). Se añadió `-host`: los contenedores (que lo necesitan) conservan 0.0.0.0, y `run.sh` local pasa 127.0.0.1 por defecto. Verificado.

Riesgos aceptados y declarados: sin autenticación ni *rate-limit* propios (en un Space público los endpoints LLM pueden drenar la cuota de Groq; mitigación: Space privado o retirar la clave), y el *warning* cosmético del tokenizador int8 en transformers 4.57.6 (comportamiento ya caracterizado por la no-regresión).

19. KC-5: identificación de «posible mismo documento» en el corpus externo

Probando el sistema, el alumno subió 2310.14735v6.pdf —un *paper* que SÍ existe en OpenAlex— con corpus externo: el sistema recuperó la obra correcta como top-1 pero solo reportó “afín de tema, similitud 55.4 %”. El diagnóstico (con datos, estilo KC) encontró tres causas de diseño y una debilidad medida:

1. La capa federada compara fragmentos del *cuerpo* contra el *abstract* (~250 palabras): `sim` ~0.55 es la respuesta correcta a esa comparación; el “100 %” es inalcanzable por construcción (no hay *full-text*).
2. `_frags_contenido` salta el primer 12 % del documento (diseño anti-portada UAM) —que en un *paper* es justo la zona título+abstract, la única que daría ~0.95.
3. No existía señal de “mismo documento” para el corpus externo (el interno sí la tiene: autocopia).
4. **Debilidad medida:** la búsqueda de OpenAlex es tipo AND —un pasaje literal de 350 caracteres devuelve `count: 0` aunque la obra exista (el abstract verbatim dio 0 resultados; 14 *keywords* la encontraron como #1 de 5,336).

Implementación (`federado.py` + frontend): (i) *zona identificadora* = título aproximado + abstract propios, saltando el tramo de autores/afiliaciones (medido: incluirlo expulsaba a la obra del top-*k* y diluía la similitud a 0.59); (ii) *query* corta de *keywords* construida solo del abstract, usada también como *fallback* en `_openalex` cuando el pasaje literal devuelve vacío; (iii) comparación MiniLM zona↔(título+abstract) de cada candidato con umbral `UMBRAL_MISMO_DOC= 0.90` (separación medida: obra verdadera **0.947** vs. siguiente 0.662); (iv) *banner* prominente en la interfaz + marca en la fuente + resumen prioritario; (v) reintentos ante 429/5xx del *pool polite*. La identificación se declara honesta: por título+abstract, no por comparación de texto completo.

Robustez cazada de paso · el singleton federado quedaba envenenado

Una carga a medias de `_cargar()` (reproducido: `NotImplementedError` de tensores *meta* cuando MarianMT cargaba en paralelo con el *prewarm* del índice de 4 GB) dejaba `_trad` asignado con `_verif/_minilm` en `None`: el federado moría para siempre. Se añadió limpieza total en el `except` → el siguiente intento recarga de cero (auto-recuperación).

Gap de despliegue cazado en la re-verificación · beto-pawsx no viajaba

El manifiesto asumía que `beto-pawsx` (verificador en-en del federado) “se descarga del Hub” —falso: es un *fine-tune* local de la semana 6 que no existe en ningún hub. Sin él, el federado habría arrancado muerto en producción. Corregido en tres puntos: `sync_dataset.sh` lo añade al dataset (dry-run: 10 artefactos, 8.3 GB, 0 faltantes), `federado.py` resuelve primero en `$MODELOS_DIR` (la ruta del Space) y el centinela de `start.sh` lo vigila.

20. Despliegue a producción del sistema v6 (autorizado por el asesor)

Con la autorización del asesor se ejecutó el despliegue completo: subida de los **10 artefactos (8.3 GB)** al dataset (el *dedup* de HF hizo casi instantáneos los seis que ya estaban del v5), preparación del repo del Space (verificado pre-push: cero secretos, sin `.env`, los diez módulos compilan) y *push* —sin caída de servicio: HF construye la imagen nueva mientras la anterior sigue sirviendo. Al ser el mismo Space del v5, los secretos y variables ya estaban configurados.

El log del primer arranque respondió las incógnitas abiertas

`CPU threads = 2` (cuota real; host reporta 16) · `AVX512-VNNI`: sí —el fix de hilos P-1 funciona en producción y **la CPU del Space SÍ tiene VNNI**, por lo que el int8 rinde allá *mejor* que en la máquina local Haswell (la medición local era cota inferior, como se predijo). La capa de oraciones cargó sus 4,144,019 vectores en **49 s** (vs. 160–200s local) y `/api/estado` confirmó las cinco capas activas con ambos backends int8: **la RAM alcanzó con ORACIONES=1**.

Tres hallazgos en producción, corregidos el mismo día (los tres invisibles desde el entorno de desarrollo):

#	Síntoma	Causa raíz	Fix (commit)
1	5 cargas repetidas de MarianMT en el log	carrera de carga perezosa sin <i>lock</i> (traductor bilingüe)	<i>double-checked locking</i> en los dos niveles (ad8c9d6)
2	federado muerto (<code>ImportError</code>)	<code>sentencepiece</code> ausente de <code>requirements</code> (el <code>venv</code> local lo tenía instalado de semanas previas)	pin <code>sentencepiece==0.2.1</code> (3778413)
3	OpenAlex rechazaba al Space (<code>HTTPError</code> ; Groq sí respondía)	filtro anti-bot contra IPs de <i>datacenter</i> con <i>User-Agent</i> anónimo	UA identificado con URL del proyecto + <code>mailto</code> + reintentos (dce6704)

Tras el tercer fix, la **verificación final en producción** reprodujo el escenario exacto que motivó KC-5: `2310.14735v6.pdf` contra el corpus externo $\rightarrow$ 12 obras, 1 posible solapamiento, y **documento identificado** («Unleashing the potential of prompt engineering...», 2023) con **sim 0.947**.

20.1. Rendimiento medido en producción

Ante el reporte de lentitud del alumno se perfiló el análisis de PDF por fases (sondeo de `/api/progreso`):

Prueba (producción, vCPU+VNNI)	2	Tiempo	Local	Nota
PDF chico (766 chars, 4 frags)		12 s	~3 s	normal
UAM0534 (48.7k chars, 181 frags)	181	131 s	112 s	solo 1.17× más lento ; titular 97.24 % reproduce el A/B local
Modo 1 bilingüe EN $\leftrightarrow$ ES		8.9 s	—	detecta idiomas, traduce, prob 0.889
Modo “ambos” (prueba E)		28 s	—	CBI 0 % correcto + 11 obras externas

Perfil del caso mediano: fragmentación 11 s · *embeddings* 32 s · búsqueda 10 s · análisis 11 s · **clasificación 65 s (fase dominante, igual que en local)**. La lentitud reportada se explica: el *primer* análisis tras el redespliegue paga dentro de su tiempo las cargas perezosas (oraciones ~49 s; modelos del federado si usa externo/ambos), y el tiempo escala ~370 chars/s. Regla práctica: documento chico = segundos · 50k chars $\approx$ 2 min · tesis completa $\approx$ 5–10 min. Para bajar a 10–20 s por tesis, la ruta ya investigada en I-1 (ZeroGPU o CPU de pago) es una decisión de presupuesto, no de código.

20.2. PWA y export a PDF verificados con navegador real

Con un *harness* CDP propio (Node + WebSocket nativo, Chrome *headless*) contra producción: la **PWA** cumple los criterios de instalación (manifest sin errores, *display standalone*, *service worker* activo sobre HTTPS, los cuatro activos en 200); y el **export a PDF** se probó con un análisis real disparado por la interfaz (pegar texto $\rightarrow$ analizar $\rightarrow$ reporte de 13 tarjetas) seguido de la misma ruta del botón (`window.print()`): **PDF de 7 páginas** con el score, las fuentes, la telemetría de las tres capas y el resumen ejecutivo de la IA; el CSS de impresión oculta el formulario y no recorta el documento. Hallazgo de método: los reportes *precomputados* no existen en el Space (leen `semana_7/resultados/`, que no viaja al despliegue), así que la prueba válida es siempre un análisis en caliente.

Cierre. Con esto, las pruebas de producción quedan completas (Modo 1 · bilingüe · fragmento federado · tesis CBI/externo/ambos · identificación · *flags* · IA · PWA · export PDF) y el sistema v6 queda **desplegado, verificado y caracterizado**. Lecciones para el proyecto: (1) las dependencias invisibles del entorno de desarrollo (`sentencepiece`, como antes `sacremoses`) solo se

cazan desplegando; (2) los *smoke tests* de despliegue deben ejercitar **cada capa perezosa** —el arranque “verde” no ejecuta el federado; (3) el mismo patrón de carrera de carga perezosa apareció tres veces en el proyecto (índices, traductor, federado): todo cargador perezoso nace con *lock* de ahora en adelante.

Fuentes de la investigación I-1

- Hardware de Spaces (CPU Basic = 2 vCPU/16 GB; tabla de precios): <https://huggingface.co/docs/hub/spaces-overview>
- ZeroGPU (GPU por llamada, solo Gradio, cuotas): <https://huggingface.co/docs/hub/en/spaces-zerogpu>
- Inference Endpoints (scale-to-zero): <https://huggingface.co/docs/inference-endpoints/index>
- Inference Providers (serverless por petición): <https://huggingface.co/blog/inference-providers>
- HF Jobs (cómputo efímero por segundo): <https://huggingface.co/docs/hub/jobs>
- Aceleración ONNX/int8 en sentence-transformers (3.08×): https://sbert.net/docs/sentence_transformer/usage/efficiency.html
- ONNX Runtime + BERT int8 (2.9×, Microsoft): <https://medium.com/microsoftazure/faster-and-smaller-quant>
- BERT en AWS Lambda (contenedor Docker): <https://github.com/philschmid/serverless-bert-huggingface-aws>
- SageMaker Serverless Inference (solo CPU): <https://aws.amazon.com/blogs/machine-learning/host-hugging-face>

Fuentes de la investigación I-2

- SPECTER2 (AllenAI; benchmark SciRepEval): <https://allenai.org/blog/specter2-adapting-scientific-docum>
- Comparativa SciBERT/SciNCL/SPECTER2: <https://arxiv.org/html/2405.04136v1>
- OpenAlex (250M+ obras, API sin key): <https://api.openalex.org>
- Comparativa OpenAlex/Semantic Scholar/PubMed: <https://intuitionlabs.ai/articles/openalex-semantic-sch>
- APIs de literatura científica 2026 (S2 embeddings vía API; CORE 260M): <https://intuitionlabs.ai/articles/research-paper-apis-scientific-literature>
- S2ORC (81M papers): <https://www.semanticscholar.org/paper/5c5751d45e298cea054f32b392c12c61027d2fe7>
- LaBSE: <https://www.emergentmind.com/topics/language-agnostic-bert-sentence-embedding-labse>
- Multilingual E5 (reporte técnico): <https://arxiv.org/pdf/2402.05672>
- Traducir vs multilingüe (20–30×, Meta): <https://engineering.fb.com/2018/01/24/ml-applications/under-the-hood-multilingual-embeddings/>
- La Referencia (tesis de 100+ universidades): <https://www.lareferencia.info/en/component/k2/item/299-la-referencia-firma-acuerdo-redalyc>
- SciELO/Dialnet/Redalyc: <https://tesify.es/scielo-dialnet-redalyc-comparativa>

Fuentes de la investigación I-3 (T1)

- Prediction-Powered Inference (Angelopoulos et al., *Science* 2023) y PPI++: <https://arxiv.org/pdf/2311.01453>
- StratPPI — evaluación híbrida de LLMs: <https://arxiv.org/pdf/2406.04291>
- Active Testing (Kossen et al., ICML 2021): <https://proceedings.mlr.press/v139/kossen21a.html>
- Active Testing para LLMs (asignación de Neyman, 2026): <https://arxiv.org/pdf/2605.10075>

- mDeBERTa-v3-XNLI (entailment multilingüe): <https://huggingface.co/MoritzLaurer/mDeBERTa-v3-base-mnli-xnli>
- Precedente interno: `s7 consenso_validacion.json` (unánime = 100 % sobre 40 pares) — refutado como regla general en la §9.